Overview of regression techniques

Analysing Ecological Data with Detection Error

Péter Sólymos

2026-04-19

Table of contents

Preamble **1**

Regression analysis **2**
- Main effects . . . 9
- Nonlinear effects . . . 10
- Categorical variables . . . 11
- Interactions . . . 12

Generalized Linear Models **19**
- Count regression models . . . 45
  - Poisson null model . . . 47
- Main effects . . . 49
  - Non-linear effects . . . 51
  - Categorical variables . . . 53
  - Multiple main effects . . . 55
  - Interactions . . . 56
- Different error distributions . . . 60
- Mixed models . . . 66

Summary **68**

Preamble

```r
suppressPackageStartupMessages({
    library(pscl)
    library(lme4)
    library(mgcv)
    library(dplyr)
    library(ggplot2)
    theme_set(theme_light())
    library(visreg)
})

if (!interactive()) {
    d <- readRDS("../data/ab_birds_site_year.rds")
} else {
    d <- readRDS("data/ab_birds_site_year.rds")
}

spp <- "yellowrumped_warbler"

x <- data.frame(y = d$counts[, spp], d$vars)

x$yday <- as.POSIXlt(x$fielddate)$yday / 365
x$hr <- x$fieldtime / 24
x <- x |>
    dplyr::filter(
        hr > 0.1,
        hr < 0.6,
        yday > 0.37,
        yday < 0.5
    )
```

Regression analysis

Almost everything in statistics can be formulated as a regression analysis problem.

We are making the following assumptions about the data:

- the data are representative sample from the population
- the data are independent of each other and are identically distributed

These are count data and hence a natural model for them is a discrete distribution such as Poisson or Negative Binomial. However, for the sake of initial analysis, we will use a Normal distribution by using the log-transformed data (after adding 0.5 to the 0's). The histogram

indicates that it might be approximately symmetric (except for the peak indicating the original zeros).

```
x$log_y <- log(x$y + 0.5)

hist(x$log_y)
```

Let us assume that the data are approximately bell shaped and a Gaussian distribution fits it well. In mathematical notation, we write this as:

$Y_i \sim Normal(\mu, \sigma^2)$

where Y_i is the random variable representing the log-transformed count at site i, μ is the mean of the distribution, and σ^2 is the variance. The observations y_i are realizations of these random variables.

```
m0 <- lm(log_y ~ 1, data = x)

summary(m0)
```

```
Call:
lm(formula = log_y ~ 1, data = x)

Residuals:
    Min      1Q  Median      3Q     Max
```

```
-1.3880 -1.3880 -0.2894  1.4452  2.9558

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.69484    0.05001   13.89   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.444 on 833 degrees of freedom
```

```r
summary(m0)$sigma # residual standard deviation
```

```
[1] 1.444235
```

```r
summary(m0)$r.squared # R^2
```

```
[1] 0
```

```r
coef(m0)
```

```
(Intercept)
  0.6948428
```

```r
mean(x$log_y)
```

```
[1] 0.6948428
```

```r
mean(fitted(m0))
```

```
[1] 0.6948428
```

I The best point predictor for the population abundance at a new location is the mean of the population, namely, .

I We use the sample mean to predict the population abundance at a new location. In our data example, this turns out to be approximately exp(ˆ) = 5.37

Is that a sufficient answer for a manager? We suspect not

We can differentiate between the following types of uncertainties:

1. If we had a slightly different sample, the sample mean would be different. This is the estimation uncertainty. How different would these sample means be? This is answered by the **confidence interval**
2. Even if we knew the population mean, we still have uncertainty about the specific observation at a new location. This is because of the intrinsic uncertainty characterized by σ
3. Uncertainty in the prediction of an observation at a new location is a combination of the estimation uncertainty and the intrinsic uncertainty, the **prediction interval**
4. There is another uncertainty, that of not knowing that the true distribution is a Normal distribution, this is model uncertainty

The estimation uncertainty can be reduced by using good sampling designs and by increasing the sample size.

Question: What constitutes as a good sampling design?

Intrinsic uncertainty is unaffected by sampling design and sample size. This can be reduced only by using an appropriate model and useful auxiliary information. Model selection and choice of appropriate covariates is critical to reduce the intrinsic uncertainty.

Let us see how we can use auxiliary information in the above problem. As biologists, we know that there are a number of different habitat characteristics that affect the population abundance.

1. Can we use these habitat characteristics to reduce the intrinsic variability?
2. How do we decide which habitat characteristics are the most important?

There are a few exploratory techniques that we find useful in making these decisions. We will describe them and illustrate their use in the data analysis.

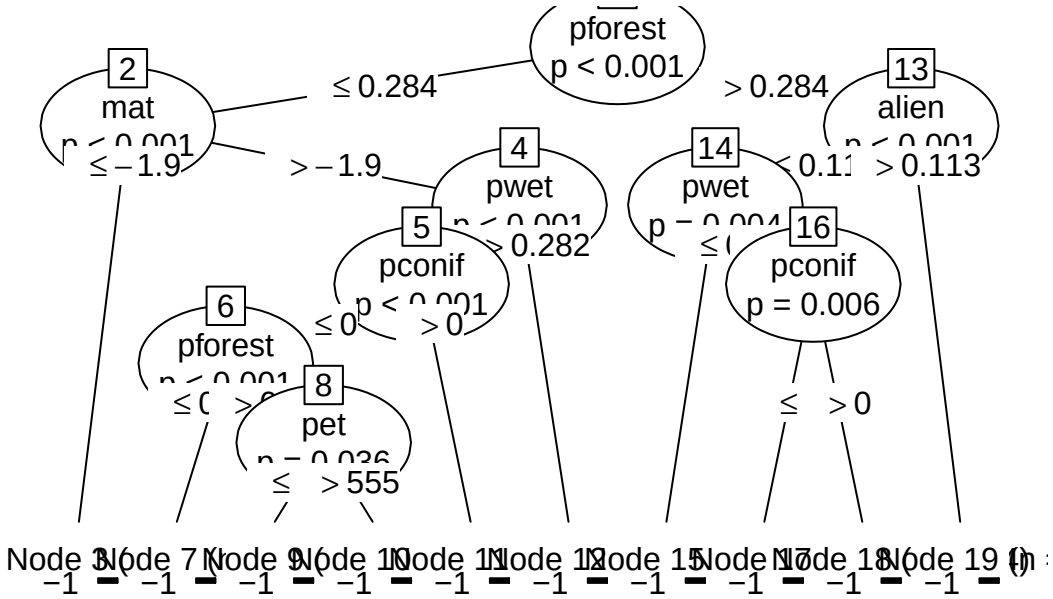
Classification and Regression Trees (CART): This technique is very useful in ranking different covariates in terms of their use in prediction. The actual implementation of the technique involves a number of decisions that are made automatically (black-box) in the algorithm. These may or may not be appropriate for every data set. We use this technique for screening the covariates.

```
ct <- x |>
    dplyr::select(
        -y,
        -site, -year, -site_year,
        -fielddate, -fieldtime,
        -nrname, -nsrname
    ) |>
    partykit::ctree(
        log_y ~ .,
```

```r
        data = _
    )

ct
```

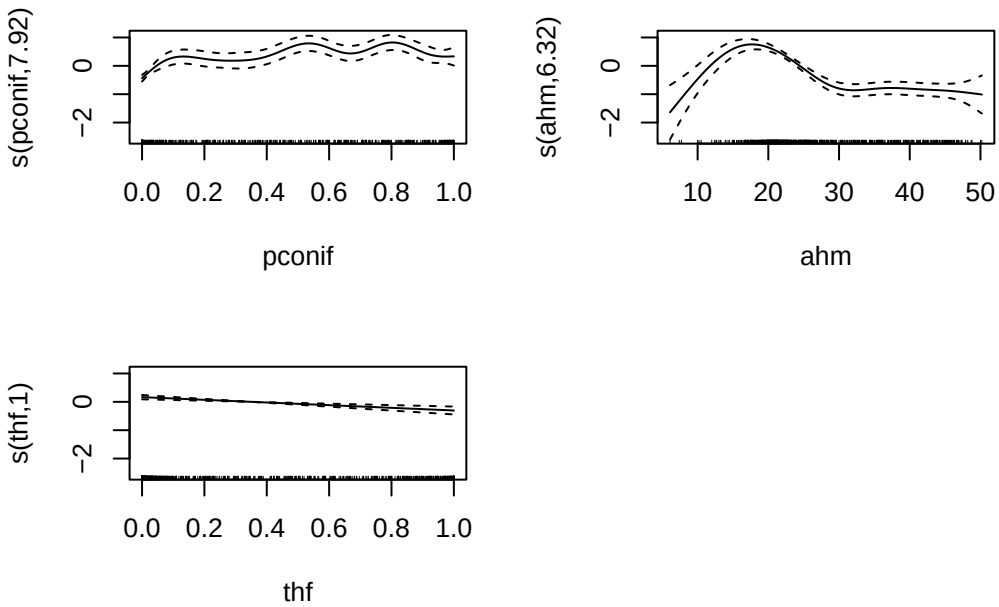
```
Model formula:
log_y ~ longitude + latitude + ahm + pet + ffp + map + mat +
    mcmt + mwmt + pforest + pconif + pwet + poldfor + cult +
    urbind + softlin + hardlin + cc + alien + succ + thf + wind +
    precip + yday + hr

Fitted party:
[1] root
|   [2] pforest <= 0.28443
|   |   [3] mat <= -1.9: 1.958 (n = 13, err = 8.1)
|   |   [4] mat > -1.9
|   |   |   [5] pwet <= 0.28198
|   |   |   |   [6] pconif <= 0.00014
|   |   |   |   |   [7] pforest <= 0.06076: -0.678 (n = 281, err = 6.1)
|   |   |   |   |   [8] pforest > 0.06076
|   |   |   |   |   |   [9] pet <= 555: 0.142 (n = 11, err = 8.6)
|   |   |   |   |   |   [10] pet > 555: -0.557 (n = 28, err = 4.5)
|   |   |   |   [11] pconif > 0.00014: 0.094 (n = 26, err = 17.8)
|   |   |   [12] pwet > 0.28198: 0.766 (n = 23, err = 46.4)
|   [13] pforest > 0.28443
|   |   [14] alien <= 0.11275
|   |   |   [15] pwet <= 0: 1.032 (n = 44, err = 78.8)
|   |   |   [16] pwet > 0
|   |   |   |   [17] pconif <= 0: 0.338 (n = 8, err = 10.7)
|   |   |   |   [18] pconif > 0: 1.861 (n = 354, err = 404.7)
|   |   [19] alien > 0.11275: 0.688 (n = 46, err = 56.7)

Number of inner nodes:     9
Number of terminal nodes: 10
```

```r
plot(ct)
```

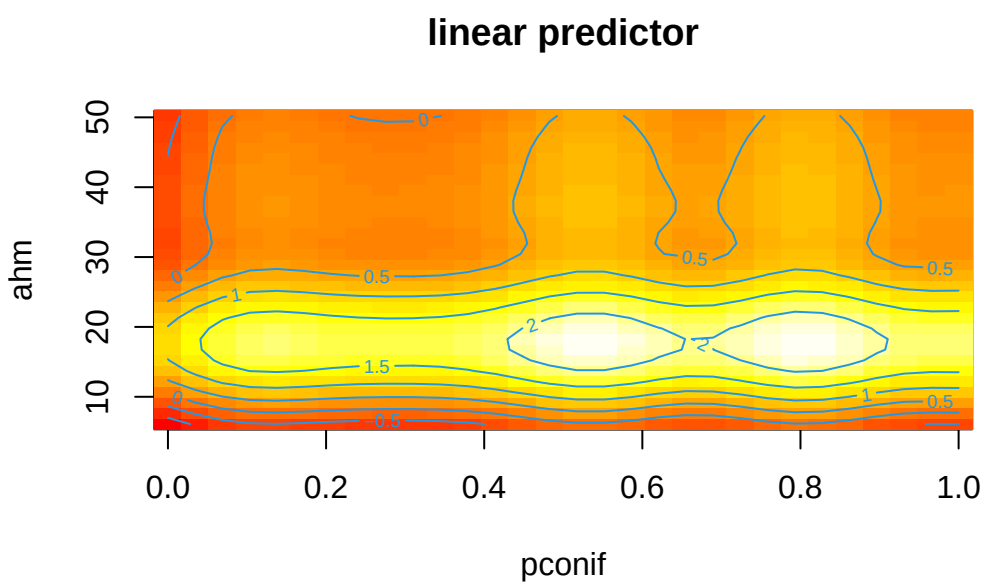


Generalized Additive Models (GAM): This technique is useful in determining if there are any non-linearities in the relationship between the covariates and the response variable.

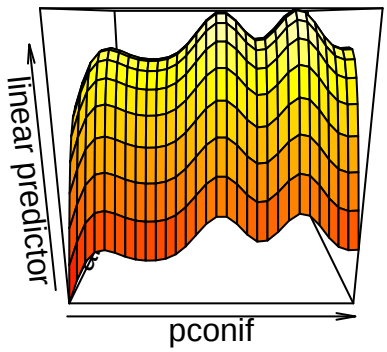
```
gm <- mgcv::gam(
    log_y ~ s(pconif) + s(ahm) + s(thf),
    data = x
)

plot(gm, pages = 1)
```




```
mgcv::vis.gam(gm, c("pconif", "ahm"), plot.type = "contour")
```




```
mgcv::vis.gam(gm, c("pconif", "ahm"), plot.type = "persp")
```



We now analyze the data using the most important covariate according to the CART analysis. We will see that using auxiliary information reduces the prediction error.

1. We keep on adding the covariates until the prediction error is sufficiently small.
2. Adding the covariates decreases the intrinsic variability but increases the estimation uncertainty. The model selection approach has to balance between these two trade-offs. A commonly used method for model selection is based on the Akaike Information Criterion (AIC) and its variants.
3. This can be done in a forward or backward fashion. The two answers do not always match (in fact, they seldom match).

Main effects

```
m1 <- lm(log_y ~ pconif, data = x)
m2 <- lm(log_y ~ pconif + ahm, data = x)
m3 <- lm(log_y ~ pconif + ahm + thf, data = x)

anova(m0, m1, m2, m3)
```

```
Analysis of Variance Table

Model 1: log_y ~ 1
Model 2: log_y ~ pconif
Model 3: log_y ~ pconif + ahm
Model 4: log_y ~ pconif + ahm + thf
  Res.Df     RSS Df Sum of Sq       F    Pr(>F)
1    833 1737.48
2    832 1076.17  1    661.32 685.081 < 2.2e-16 ***
3    831  874.50  1    201.67 208.915 < 2.2e-16 ***
4    830  801.21  1     73.29  75.924 < 2.2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
AIC(m0, m1, m2, m3)
```

```
   df      AIC
m0  2 2982.912
m1  3 2585.399
m2  4 2414.336
m3  5 2343.336
```

```r
MuMIn::model.sel(m0, m1, m2, m3)
```

```
Model selection table
    (Intrc) pconf      ahm     thf df    logLik   AICc  delta weight
m3 2.330000 1.131 -0.06183 -0.9262  5 -1166.668 2343.4   0.00      1
m2 2.178000 1.508 -0.07240          4 -1203.168 2414.4  70.98      0
m1 0.001609 2.544                   3 -1289.699 2585.4 242.02      0
m0 0.694800                         2 -1489.456 2982.9 639.52      0
Models ranked by AICc(x)
```

Nonlinear effects

```r
m4 <- lm(log_y ~ pconif + I(pconif^2), data = x)
m5 <- lm(log_y ~ pconif + I(pconif^2) + ahm, data = x)
m6 <- lm(log_y ~ pconif + I(pconif^2) + ahm + thf, data = x)

MuMIn::model.sel(m0, m1, m2, m3, m4, m5, m6)
```

```
Model selection table
     (Intrc) pconf      ahm     thf pconf^2 df    logLik   AICc  delta weight
m6  1.772000 4.260 -0.05120 -0.7006  -3.317  6 -1141.002 2294.1   0.00      1
m5  1.523000 5.331 -0.05577          -4.175  5 -1161.752 2333.6  39.47      0
m3  2.330000 1.131 -0.06183 -0.9262          5 -1166.668 2343.4  49.30      0
m2  2.178000 1.508 -0.07240                  4 -1203.168 2414.4 120.28      0
m4 -0.213200 7.500                   -5.772  4 -1214.160 2436.4 142.26      0
m1  0.001609 2.544                           3 -1289.699 2585.4 291.32      0
m0  0.694800                                 2 -1489.456 2982.9 688.82      0
Models ranked by AICc(x)
```

Categorical variables

```r
# reference (1st) level is Boreal
m7 <- lm(log_y ~ nrname, data = x)
summary(m7)
```

```
Call:
lm(formula = log_y ~ nrname, data = x)

Residuals:
     Min       1Q   Median       3Q      Max 
-2.39851 -0.07697 -0.00504  0.72238  2.30634 

Coefficients:
                      Estimate Std. Error t value Pr(>|t|)    
(Intercept)            1.41768    0.04952  28.631  < 2e-16 ***
nrnameCanadian Shield  0.28768    0.23383   1.230  0.21893    
nrnameFoothills        0.28609    0.12655   2.261  0.02404 *  
nrnameGrassland       -2.10579    0.08510 -24.743  < 2e-16 ***
nrnameParkland        -2.03386    0.12587 -16.159  < 2e-16 ***
nrnameRocky Mountain  -0.86182    0.26848  -3.210  0.00138 ** 
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.022 on 828 degrees of freedom
Multiple R-squared:  0.5023,	Adjusted R-squared:  0.4993 
F-statistic: 167.1 on 5 and 828 DF,  p-value: < 2.2e-16
```

```r
MuMIn::model.sel(m0, m1, m2, m3, m4, m5, m6, m7)
```

```
Model selection table 
     (Intrc) pconf      ahm     thf pconf^2 nrnam df    logLik   AICc  delta
m6  1.772000 4.260 -0.05120 -0.7006  -3.317        6 -1141.002 2294.1   0.00
m5  1.523000 5.331 -0.05577          -4.175        5 -1161.752 2333.6  39.47
m3  2.330000 1.131 -0.06183 -0.9262                5 -1166.668 2343.4  49.30
m7  1.418000                                   +   7 -1198.520 2411.2 117.07
m2  2.178000 1.508 -0.07240                        4 -1203.168 2414.4 120.28
m4 -0.213200 7.500                   -5.772        4 -1214.160 2436.4 142.26
m1  0.001609 2.544                                 3 -1289.699 2585.4 291.32
```

```
m0  0.694800                                        2 -1489.456 2982.9 688.82
   weight
m6      1
m5      0
m3      0
m7      0
m2      0
m4      0
m1      0
m0      0
Models ranked by AICc(x)
```

Interactions

```r
m8 <- lm(log_y ~ pconif * ahm, data = x)
summary(m8)
```

```
Call:
lm(formula = log_y ~ pconif * ahm, data = x)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.2350 -0.7104  0.0790  0.6855  2.3325 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)  2.334542   0.160104  14.581  < 2e-16 ***
pconif      -2.562286   0.572240  -4.478 8.60e-06 ***
ahm         -0.078452   0.005141 -15.259  < 2e-16 ***
pconif:ahm   0.206672   0.028382   7.282 7.65e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9952 on 830 degrees of freedom
Multiple R-squared:  0.5269,	Adjusted R-squared:  0.5252 
F-statistic: 308.1 on 3 and 830 DF,  p-value: < 2.2e-16
```

```r
MuMIn::model.sel(m0, m1, m2, m3, m4, m5, m6, m7, m8)
```

```
Model selection table
       (Int)    pcn      ahm     thf  pcn^2 nrn ahm:pcn df    logLik   AICc
m6  1.772000  4.260 -0.05120 -0.7006 -3.317              6 -1141.002 2294.1
m5  1.523000  5.331 -0.05577         -4.175              5 -1161.752 2333.6
m3  2.330000  1.131 -0.06183 -0.9262                     5 -1166.668 2343.4
m8  2.335000 -2.562 -0.07845                     0.2067  5 -1177.344 2364.8
m7  1.418000                                 +           7 -1198.520 2411.2
m2  2.178000  1.508 -0.07240                             4 -1203.168 2414.4
m4 -0.213200  7.500                  -5.772              4 -1214.160 2436.4
m1  0.001609  2.544                                      3 -1289.699 2585.4
m0  0.694800                                             2 -1489.456 2982.9
    delta weight
m6   0.00      1
m5  39.47      0
m3  49.30      0
m8  70.65      0
m7 117.07      0
m2 120.28      0
m4 142.26      0
m1 291.32      0
m0 688.82      0
Models ranked by AICc(x)
```

Notice that as model complexity increases, the estimation uncertainty increases as well. We have to balance between the two. AIC and its variants are commonly used for this purpose, but they are not the only method. We can also use cross-validation to estimate the prediction error of the models and select the one with the lowest error.

```r
summary(m0)
```

```
Call:
lm(formula = log_y ~ 1, data = x)

Residuals:
    Min      1Q  Median      3Q     Max 
-1.3880 -1.3880 -0.2894  1.4452  2.9558 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  0.69484    0.05001   13.89   <2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.444 on 833 degrees of freedom
```

```r
confint(m0)
```

```
                2.5 %    97.5 %
(Intercept) 0.5966829 0.7930028
```

```r
summary(m6)
```

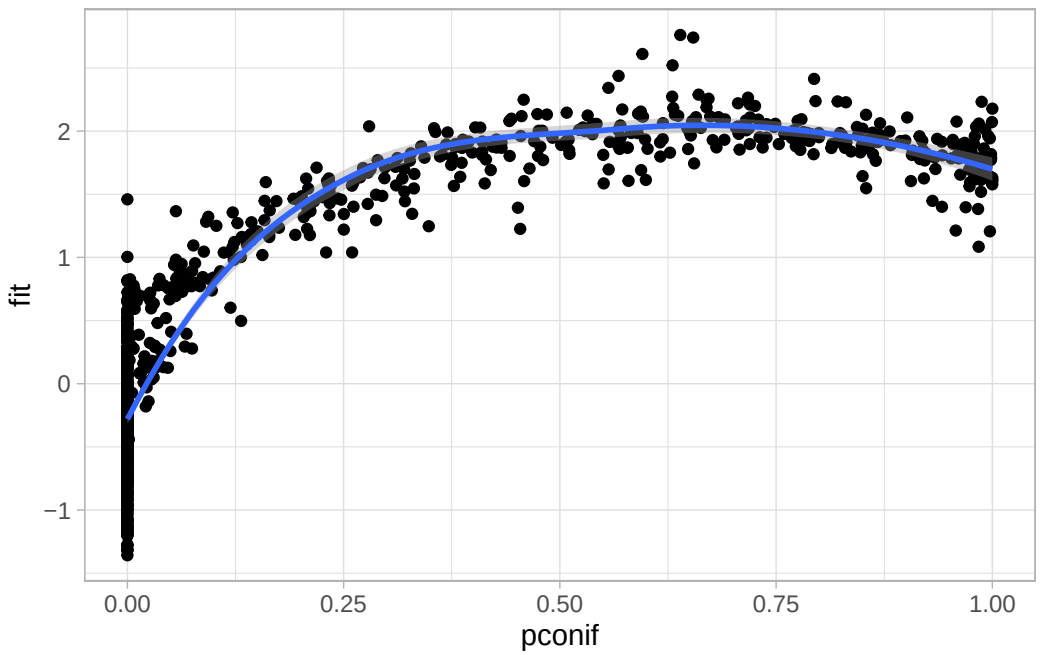
```
Call:
lm(formula = log_y ~ pconif + I(pconif^2) + ahm + thf, data = x)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4542 -0.4773  0.0683  0.6141  2.3737 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)  1.772192   0.171132  10.356  < 2e-16 ***
pconif       4.259901   0.448886   9.490  < 2e-16 ***
I(pconif^2) -3.316894   0.457218  -7.255 9.26e-13 ***
ahm         -0.051201   0.005211  -9.826  < 2e-16 ***
thf         -0.700594   0.107726  -6.503 1.36e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9533 on 829 degrees of freedom
Multiple R-squared:  0.5664,	Adjusted R-squared:  0.5643 
F-statistic: 270.7 on 4 and 829 DF,  p-value: < 2.2e-16
```

```r
confint(m6)
```

```
                  2.5 %      97.5 %
(Intercept)  1.43628813  2.10809513
pconif       3.37881425  5.14098735
I(pconif^2) -4.21433595 -2.41945225
ahm         -0.06142887 -0.04097245
thf         -0.91204165 -0.48914606
```

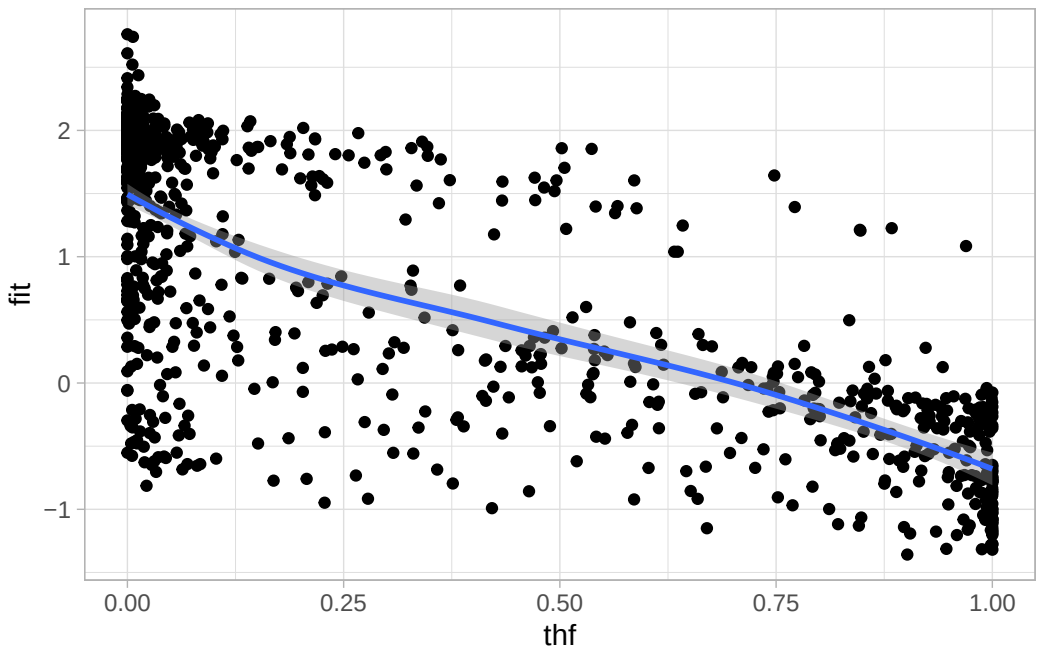
```r
# marginal effects
data.frame(x, fit = fitted(m6)) |>
    ggplot(aes(x = pconif, y = fit)) +
    geom_jitter() +
    geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



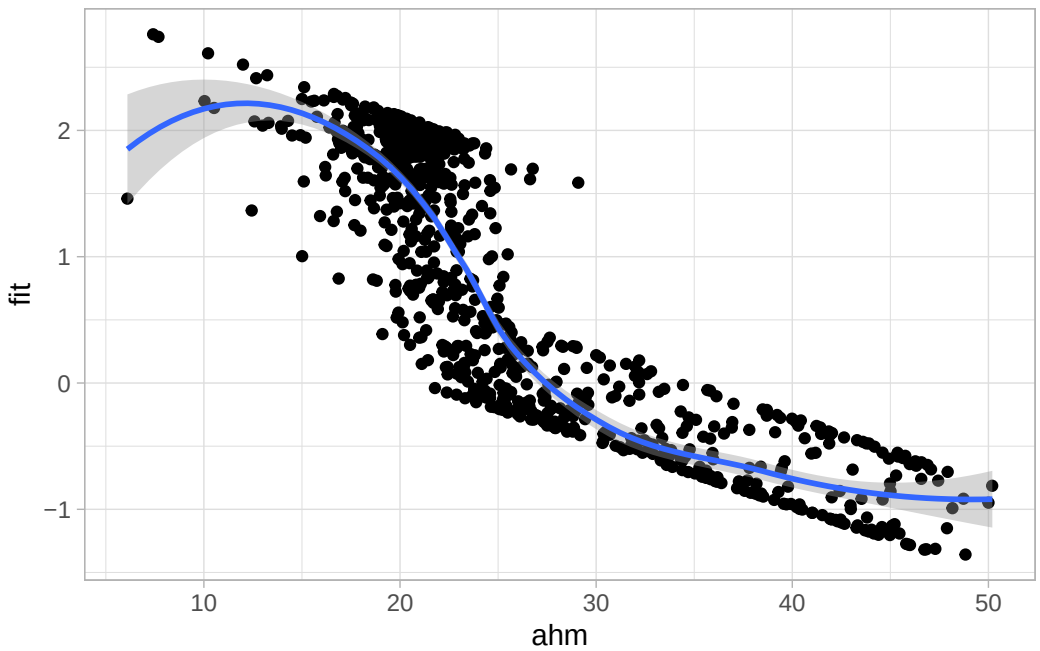

```r
data.frame(x, fit = fitted(m6)) |>
    ggplot(aes(x = thf, y = fit)) +
    geom_jitter() +
    geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

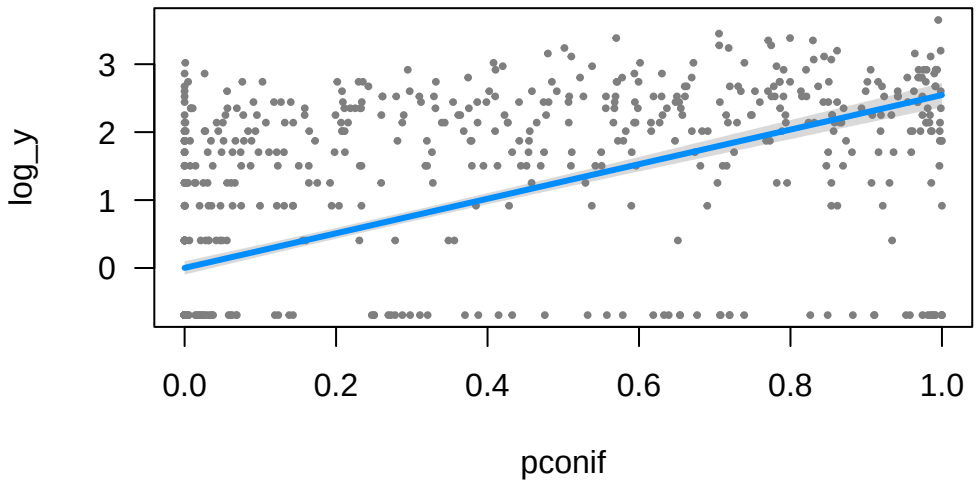



```
data.frame(x, fit = fitted(m6)) |>
    ggplot(aes(x = ahm, y = fit)) +
    geom_jitter() +
    geom_smooth()
```

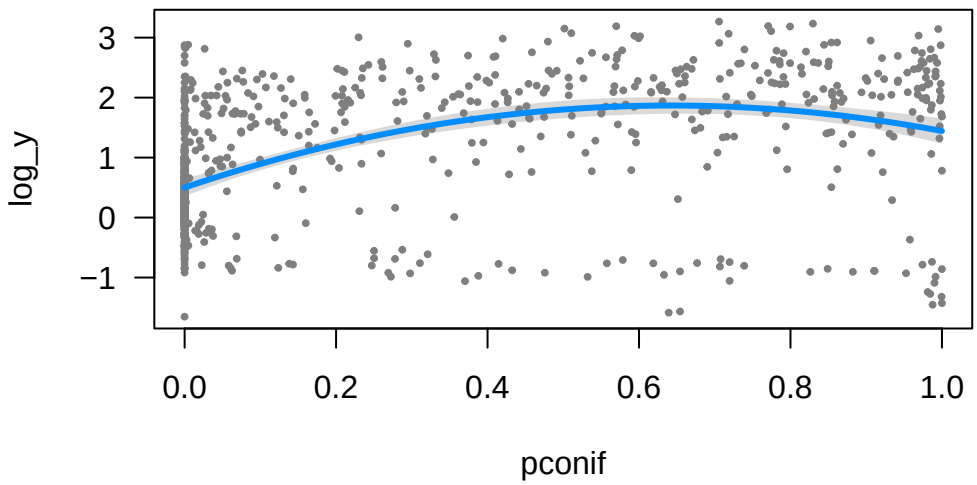
```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



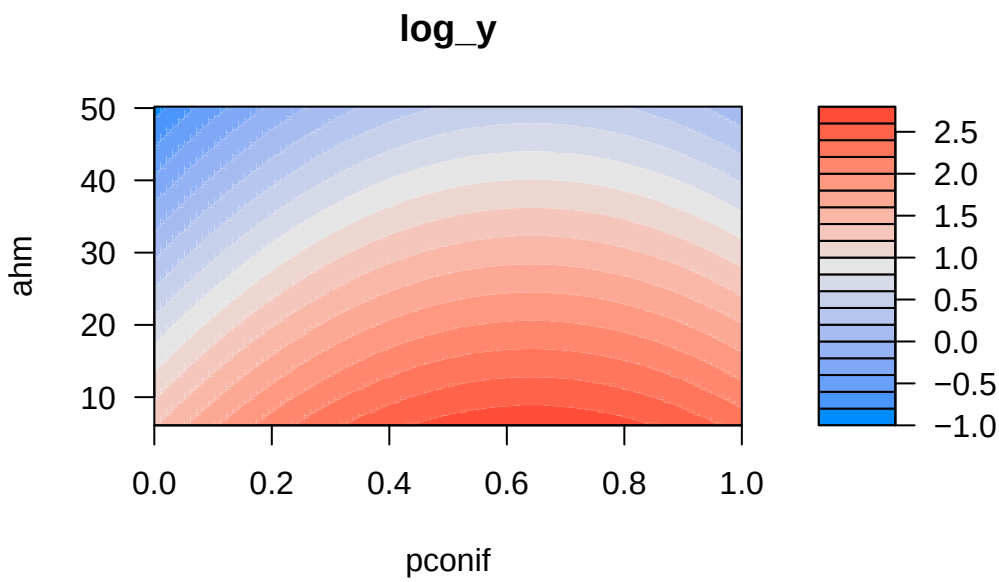

```
# partial effects
visreg::visreg(m1, "pconif")
```



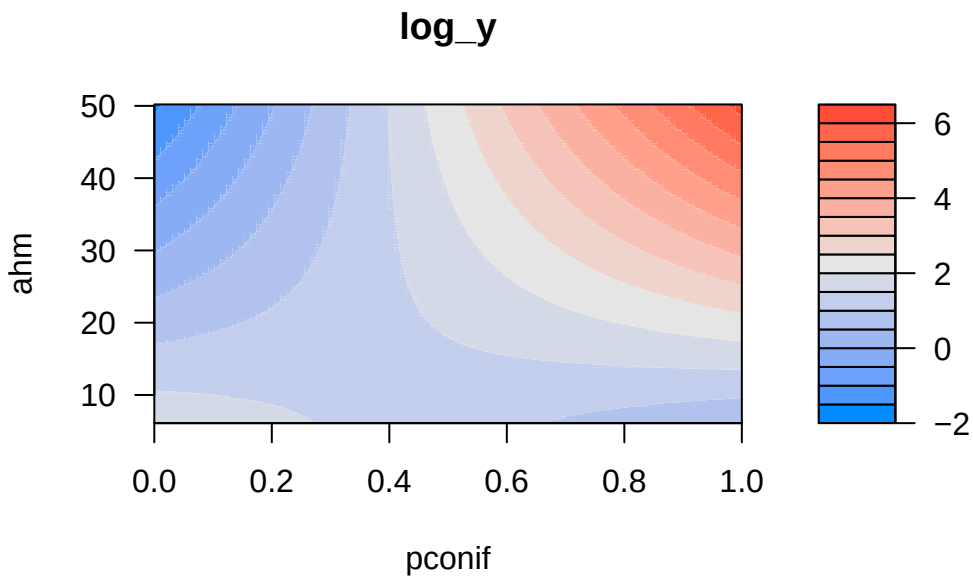

```
visreg::visreg(m6, "pconif")
```



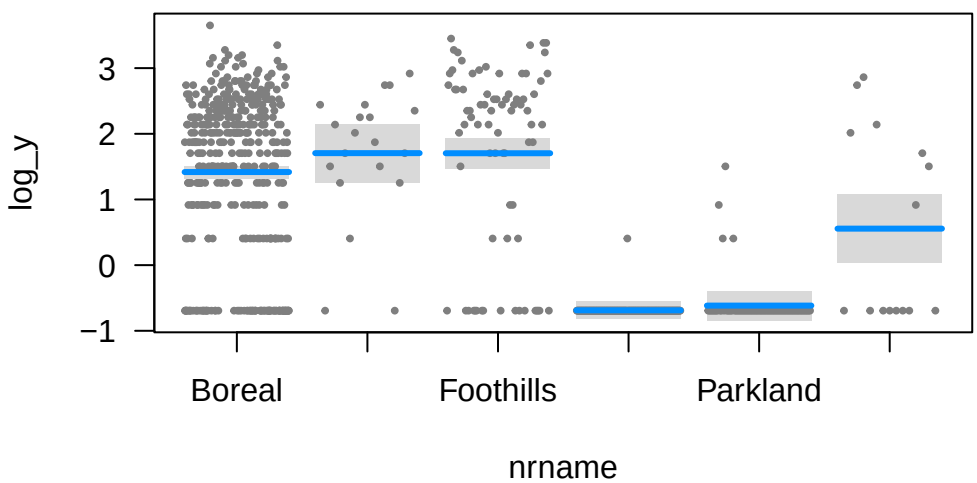

```
visreg::visreg2d(m6, "pconif", "ahm")
```




```
visreg::visreg2d(m8, "pconif", "ahm")
```




```
visreg::visreg(m7, "nrname")
```



Exercise: Play with other covariates and their interactions. Try to find a better model using AIC/AICc. Make sure that effects are sensible using `summary()` and visualizations.

Generalized Linear Models

1. Our goal in regression is to model the intrinsic variability appropriately and to reduce it using the auxiliary information.
2. Intrinsic variability in a response variable may not be appropriately modelled by a Normal distribution.

3. If the response variable is discrete, such as counts, we should use appropriate statistical model such as a discrete distribution to model intrinsic variability.
4. If the response variable is categorical, such as presence/absence of a species, we should use an appropriate distribution such as a Bernoulli distribution.

- We assume that the mean of the distribution can be modelled as a function of the covariates.
- We do NOT transform the response variable. It remains on its original scale.
- The variance of the response variable is not independent of the mean. In fact, in many cases, it varies with the mean. The assumption of constant variance in the linear regression is not necessary.

We will start by looking at the abundance data that we analyzed in the morning session. However, this time we will not transform the response variable.

The response variable is counts taking values in (0,1,2,...). A natural model to model the intrinsic variation is the Poisson distribution. One can also use Negative Binomial distribution or some such distribution. Let us see if we can use this simple distribution to predict abundance at a new location and obtain its prediction uncertainty.

If this uncertainty is small, we will not bother doing any further analysis. If this uncertainty is large, we will use auxiliary information to reduce it. The steps we follow are identical to the ones we did in the morning session.

Mean of the Poisson distribution is always positive. Hence we model the relationship between the mean of the Poisson distribution and the auxiliary information using a function that is always positive. Any such function will suffice. The most commonly used function is the log-linear function.

We use AIC to find the best fitting model that balances the estimation uncertainty and intrinsic variability.

We observe that the parameter estimates obtained by the GLM are similar but not the same as the ones we obtained by using LM on the log-transformed data. The differences between two methods are (1) No issues with 0 counts, (2) Prediction error is more appropriate on the discrete scale than the artificial continuous scale.

```r
mP0 <- glm(y ~ 1, data = x, family = poisson)
summary(mP0)
```

```
Call:
glm(formula = y ~ 1, family = poisson, data = x)

Coefficients:
```

```
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  1.51283    0.01625   93.08   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 6717.5  on 833  degrees of freedom
Residual deviance: 6717.5  on 833  degrees of freedom
AIC: 8342.8

Number of Fisher Scoring iterations: 6
```

```r
coef(mP0)
```

```
(Intercept)
   1.512832
```

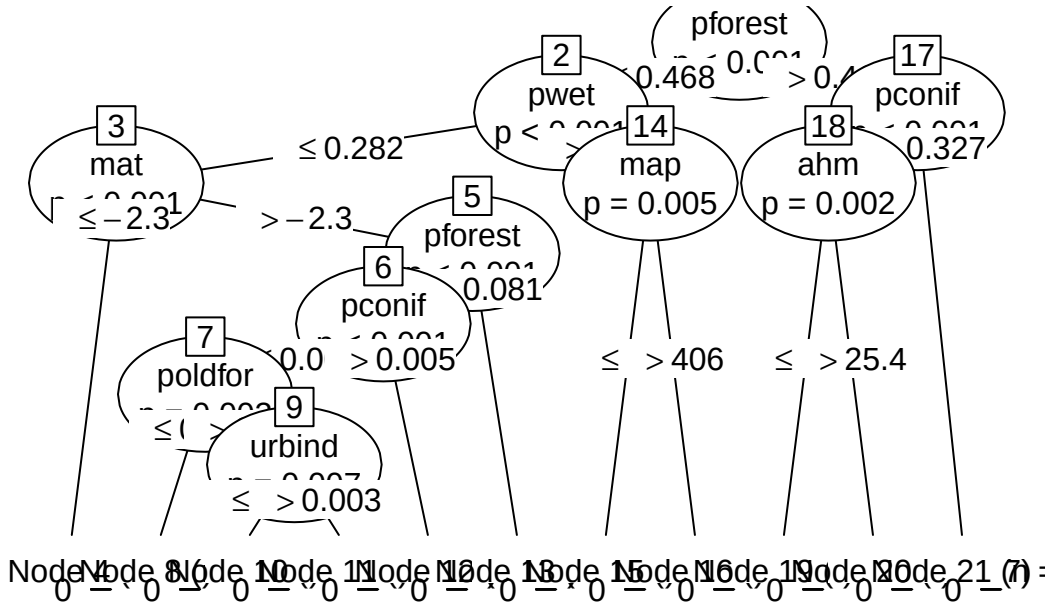
```r
confint(mP0)
```

```
Waiting for profiling to be done...

   2.5 %   97.5 %
1.480808 1.544517
```

```r
ct2 <- x |>
    dplyr::select(
        -log_y,
        -site, -year, -site_year,
        -fielddate, -fieldtime,
        -nrname, -nsrname
    ) |>
    partykit::ctree(
        y ~ .,
        data = _
    )

ct2
```

```
Model formula:
y ~ longitude + latitude + ahm + pet + ffp + map + mat + mcmt +
    mwmt + pforest + pconif + pwet + poldfor + cult + urbind +
    softlin + hardlin + cc + alien + succ + thf + wind + precip +
    yday + hr

Fitted party:
[1] root
|   [2] pforest <= 0.46808
|   |   [3] pwet <= 0.28198
|   |   |   [4] mat <= -2.3: 7.000 (n = 9, err = 182.0)
|   |   |   [5] mat > -2.3
|   |   |   |   [6] pforest <= 0.08084
|   |   |   |   |   [7] pconif <= 0.00537
|   |   |   |   |   |   [8] poldfor <= 0.00071: 0.015 (n = 260, err = 9.9)
|   |   |   |   |   |   [9] poldfor > 0.00071
|   |   |   |   |   |   |   [10] urbind <= 0.00303: 0.000 (n = 11, err = 0.0)
|   |   |   |   |   |   |   [11] urbind > 0.00303: 0.208 (n = 24, err = 6.0)
|   |   |   |   |   [12] pconif > 0.00537: 0.714 (n = 7, err = 5.4)
|   |   |   |   [13] pforest > 0.08084: 1.161 (n = 62, err = 224.4)
|   |   [14] pwet > 0.28198
|   |   |   [15] map <= 406: 2.600 (n = 20, err = 242.8)
|   |   |   [16] map > 406: 7.839 (n = 31, err = 864.2)
|   [17] pforest > 0.46808
|   |   [18] pconif <= 0.32662
|   |   |   [19] ahm <= 25.4: 5.912 (n = 113, err = 2087.1)
|   |   |   [20] ahm > 25.4: 0.706 (n = 17, err = 17.5)
|   |   [21] pconif > 0.32662: 9.507 (n = 280, err = 13370.0)

Number of inner nodes:    10
Number of terminal nodes: 11
```

```r
plot(ct2)
```




```r
mP1 <- glm(y ~ pconif, data = x, family = poisson)
summary(mP1)
```

```
Call:
glm(formula = y ~ pconif, family = poisson, data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  0.71843    0.02858   25.14   <2e-16 ***
pconif       1.94272    0.04246   45.75   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 6717.5  on 833  degrees of freedom
Residual deviance: 4588.7  on 832  degrees of freedom
AIC: 6215.9

Number of Fisher Scoring iterations: 6
```

```r
coef(mP1)
```

```
(Intercept)      pconif
  0.7184326   1.9427223
```

```r
confint(mP1)
```

```
Waiting for profiling to be done...

                2.5 %   97.5 %
(Intercept) 0.6619523 0.773991
pconif      1.8596543 2.026123
```

```r
anova(mP1, test = "Chi")
```

```
Analysis of Deviance Table

Model: poisson, link: log

Response: y

Terms added sequentially (first to last)

       Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                     833     6717.5
pconif  1   2128.8       832     4588.7 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
mP2 <- glm(y ~ pconif + I(pconif^2), data = x, family = poisson)
summary(mP2)
```

```
Call:
glm(formula = y ~ pconif + I(pconif^2), family = poisson, data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  0.27655    0.03823   7.234 4.71e-13 ***
pconif       6.19387    0.17520  35.354  < 2e-16 ***
I(pconif^2) -4.41000    0.17388 -25.363  < 2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 6717.5  on 833  degrees of freedom
Residual deviance: 3934.0  on 831  degrees of freedom
AIC: 5563.3

Number of Fisher Scoring iterations: 6
```

```r
coef(mP2)
```

```
(Intercept)      pconif I(pconif^2)
  0.2765499   6.1938714  -4.4100036
```

```r
confint(mP2)
```

```
Waiting for profiling to be done...

                 2.5 %    97.5 %
(Intercept)  0.2007631  0.350647
pconif       5.8514234  6.538248
I(pconif^2) -4.7513545 -4.069722
```

```r
anova(mP2, test = "Chi")
```

```
Analysis of Deviance Table

Model: poisson, link: log

Response: y

Terms added sequentially (first to last)

            Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                          833     6717.5
pconif       1  2128.81       832     4588.7 < 2.2e-16 ***
I(pconif^2)  1   654.65       831     3934.0 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
mP3 <- glm(y ~ pconif + I(pconif^2) + ahm, data = x, family = poisson)
summary(mP3)
```

```
Call:
glm(formula = y ~ pconif + I(pconif^2) + ahm, family = poisson,
    data = x)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  2.832392   0.110696   25.59   <2e-16 ***
pconif       4.101245   0.192901   21.26   <2e-16 ***
I(pconif^2) -3.066696   0.179634  -17.07   <2e-16 ***
ahm         -0.093277   0.004089  -22.81   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 6717.5  on 833  degrees of freedom
Residual deviance: 3392.3  on 830  degrees of freedom
AIC: 5023.6

Number of Fisher Scoring iterations: 6
```

```r
coef(mP3)
```

```
(Intercept)      pconif I(pconif^2)         ahm
 2.83239165  4.10124523 -3.06669592 -0.09327732
```

```r
confint(mP3)
```

```
Waiting for profiling to be done...

                 2.5 %      97.5 %
(Intercept)  2.6143498  3.04824286
pconif       3.7242541  4.48046069
I(pconif^2) -3.4194541 -2.71525761
ahm         -0.1012909 -0.08526569
```

```r
anova(mP3, test = "Chi")
```

```
Analysis of Deviance Table

Model: poisson, link: log

Response: y

Terms added sequentially (first to last)

            Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                          833     6717.5
pconif       1  2128.81       832     4588.7 < 2.2e-16 ***
I(pconif^2)  1   654.65       831     3934.0 < 2.2e-16 ***
ahm          1   541.67       830     3392.3 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
mP4 <- glm(y ~ pconif + I(pconif^2) + ahm + thf, data = x, family = poisson)
summary(mP4)
```

```
Call:
glm(formula = y ~ pconif + I(pconif^2) + ahm + thf, family = poisson,
    data = x)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  2.887536   0.108124   26.71   <2e-16 ***
pconif       3.195072   0.203308   15.71   <2e-16 ***
I(pconif^2) -2.352713   0.186876  -12.59   <2e-16 ***
ahm         -0.079038   0.004137  -19.11   <2e-16 ***
thf         -1.080298   0.089190  -12.11   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 6717.5  on 833  degrees of freedom
Residual deviance: 3229.3  on 829  degrees of freedom
AIC: 4862.6
```

```
Number of Fisher Scoring iterations: 6
```

```r
coef(mP4)
```

```
(Intercept)      pconif I(pconif^2)         ahm         thf
 2.88753623  3.19507239 -2.35271342 -0.07903768 -1.08029850
```

```r
confint(mP4)
```

```
Waiting for profiling to be done...

                  2.5 %      97.5 %
(Intercept)  2.67457950  3.09839271
pconif       2.79779959  3.59479554
I(pconif^2) -2.71972477 -1.98714380
ahm         -0.08716045 -0.07094823
thf         -1.25671781 -0.90703055
```

```r
anova(mP4, test = "Chi")
```

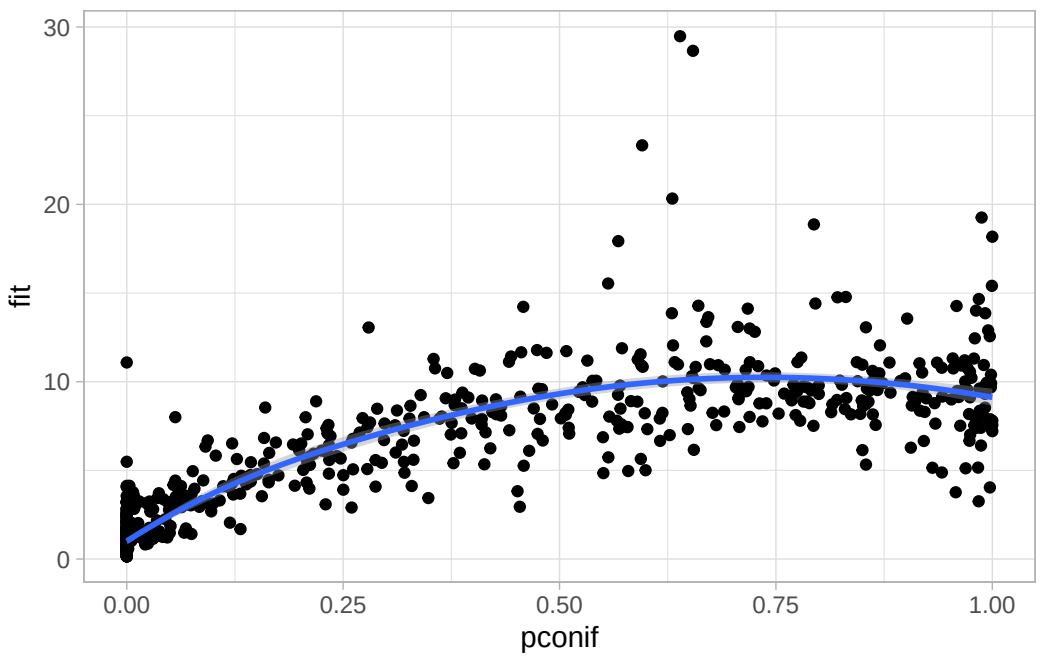
```
Analysis of Deviance Table

Model: poisson, link: log

Response: y

Terms added sequentially (first to last)

            Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                          833     6717.5
pconif       1  2128.81       832     4588.7 < 2.2e-16 ***
I(pconif^2)  1   654.65       831     3934.0 < 2.2e-16 ***
ahm          1   541.67       830     3392.3 < 2.2e-16 ***
thf          1   163.03       829     3229.3 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
MuMIn::model.sel(mP0, mP1, mP2, mP3, mP4)
```

```
Model selection table
    (Intrc) pconf pconf^2      ahm   thf df    logLik   AICc   delta weight
mP4  2.8880 3.195  -2.353 -0.07904 -1.08  5 -2426.299 4862.7    0.00      1
mP3  2.8320 4.101  -3.067 -0.09328        4 -2507.813 5023.7  161.00      0
mP2  0.2765 6.194  -4.410                 3 -2778.647 5563.3  700.65      0
mP1  0.7184 1.943                         2 -3105.974 6216.0 1353.29      0
mP0  1.5130                               1 -4170.377 8342.8 3480.09      0
Models ranked by AICc(x)
```

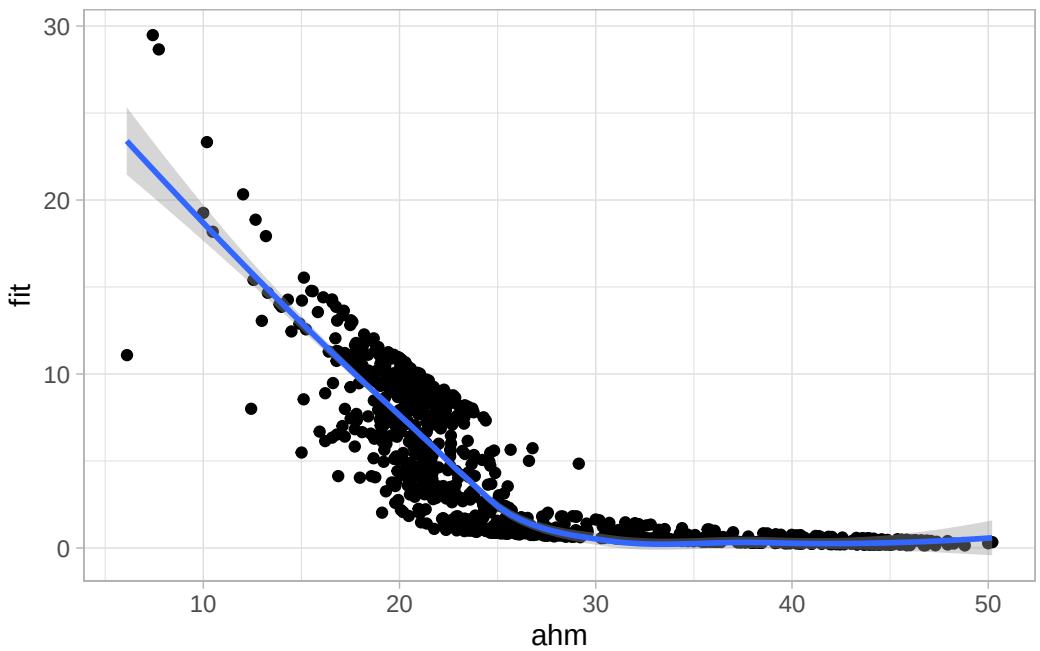
```r
# marginal effects
data.frame(x, fit = fitted(mP4)) |>
    ggplot(aes(x = pconif, y = fit)) +
    geom_jitter() +
    geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



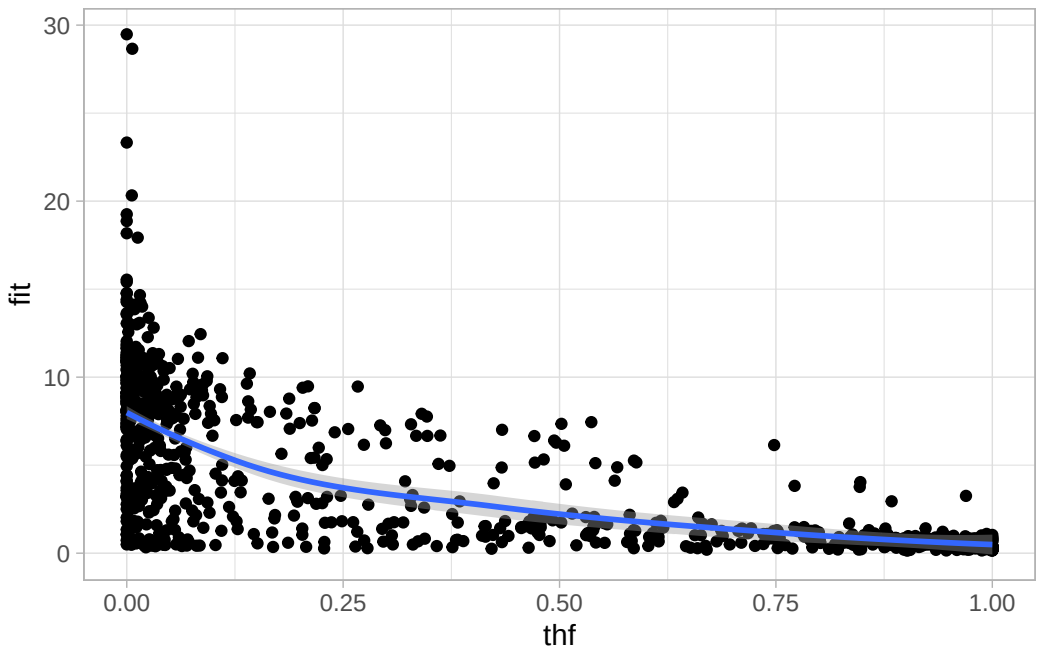
```r
data.frame(x, fit = fitted(mP4)) |>
    ggplot(aes(x = ahm, y = fit)) +
    geom_jitter() +
    geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

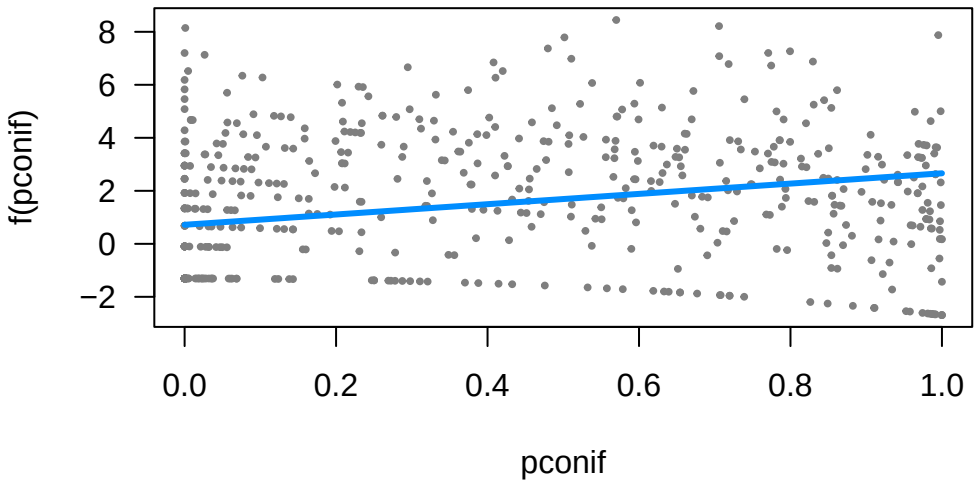



```r
data.frame(x, fit = fitted(mP4)) |>
    ggplot(aes(x = thf, y = fit)) +
    geom_jitter() +
    geom_smooth()
```

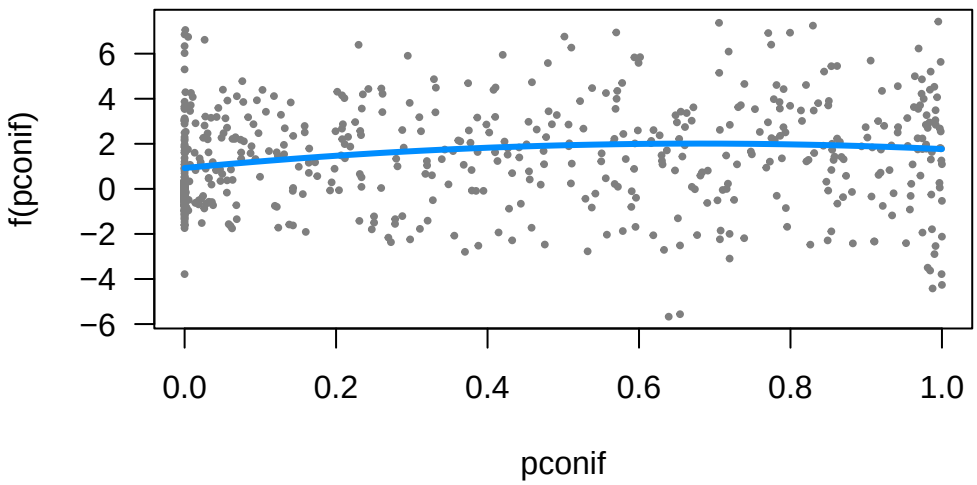
```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



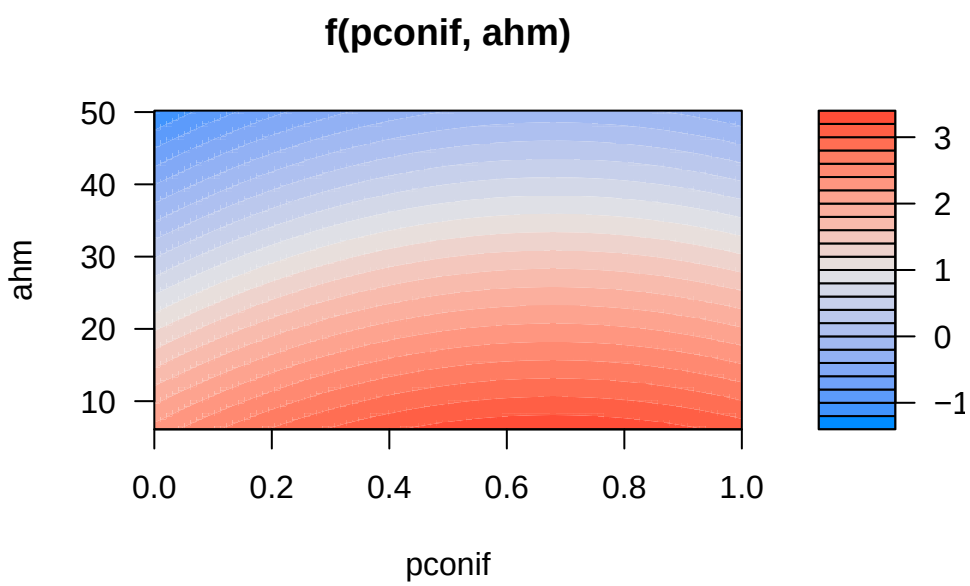

```
# partial effects
visreg::visreg(mP1, "pconif")
```




```
visreg::visreg(mP4, "pconif")
```




```
visreg::visreg2d(mP4, "pconif", "ahm")
```



Now let us use the same data but now instead of observing the actual counts, suppose we only noted 0's and 1's where 1 indicates that the count was anything other than 0. Thus, we have turned the count data into a binary data. These kinds of data are not quite common in practice. How should we analyze these data?

We follow exactly the same steps as before. 1. We fit the model without any auxiliary information. How good are the predictions? 1. Prediction accuracy for binary data is tricky to define. A common measure of the prediction accuracy is based on how often we predict 1's as 1's (Sensitivity) and 0's and 0's (Specificity). 1. We use the Receiver Operating Characteristic (ROC) curve to show the predictive ability. The area under the ROC curve is a measure

of predictive ability of a classification model. We want this area to be close to 1. Are the results comparable to LM and Poisson regression? Notice that we lose substantial amount of information when we go from count data to binary data.

```
x$y01 <- ifelse(x$y > 0, 1, 0)
mB0 <- glm(y01 ~ 1, data = x, family = binomial)
summary(mB0)
```

```
Call:
glm(formula = y01 ~ 1, family = binomial, data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  0.06717    0.06929   0.969    0.332

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1155.2  on 833  degrees of freedom
Residual deviance: 1155.2  on 833  degrees of freedom
AIC: 1157.2

Number of Fisher Scoring iterations: 3
```

```
coef(mB0)
```

```
(Intercept)
 0.06717153
```

```
confint(mB0)
```

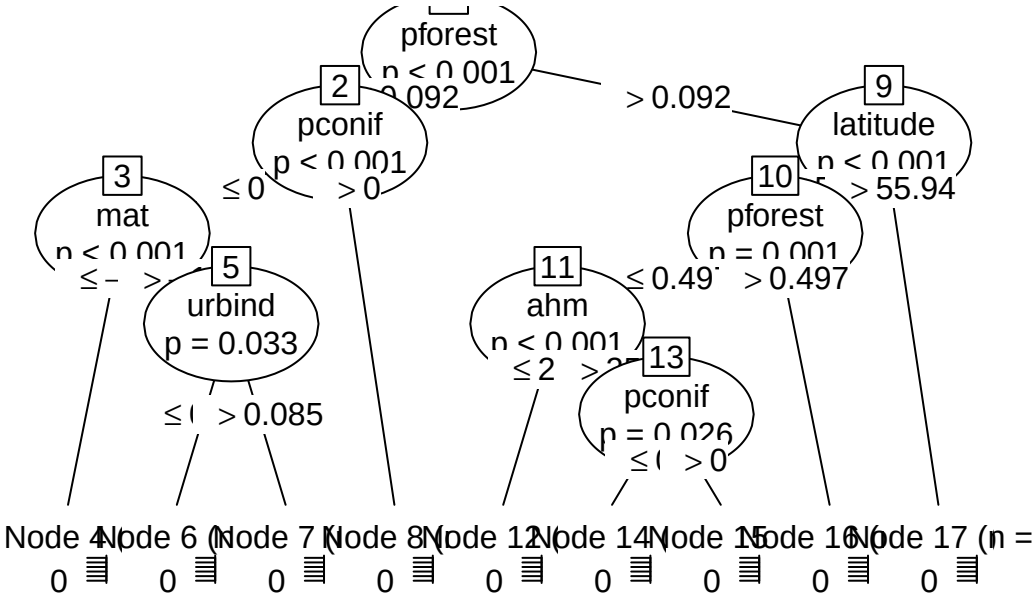
```
Waiting for profiling to be done...

      2.5 %      97.5 %
-0.06859008  0.20313997
```

```
ct3 <- x |>
    dplyr::select(
        -log_y, -y,
        -site, -year, -site_year,
```

```r
        -fielddate, -fieldtime,
        -nrname, -nsrname
    ) |>
    partykit::ctree(
        y01 ~ .,
        data = _
    )

ct3
```

```
Model formula:
y01 ~ longitude + latitude + ahm + pet + ffp + map + mat + mcmt +
    mwmt + pforest + pconif + pwet + poldfor + cult + urbind +
    softlin + hardlin + cc + alien + succ + thf + wind + precip +
    yday + hr

Fitted party:
[1] root
|   [2] pforest <= 0.09204
|   |   [3] pconif <= 3e-05
|   |   |   [4] mat <= -1.1: 0.429 (n = 7, err = 1.7)
|   |   |   [5] mat > -1.1
|   |   |   |   [6] urbind <= 0.08517: 0.011 (n = 280, err = 3.0)
|   |   |   |   [7] urbind > 0.08517: 0.143 (n = 14, err = 1.7)
|   |   [8] pconif > 3e-05: 0.611 (n = 18, err = 4.3)
|   [9] pforest > 0.09204
|   |   [10] latitude <= 55.9396
|   |   |   [11] pforest <= 0.49672
|   |   |   |   [12] ahm <= 25.6: 0.650 (n = 40, err = 9.1)
|   |   |   |   [13] ahm > 25.6
|   |   |   |   |   [14] pconif <= 0.00014: 0.000 (n = 20, err = 0.0)
|   |   |   |   |   [15] pconif > 0.00014: 0.429 (n = 7, err = 1.7)
|   |   |   [16] pforest > 0.49672: 0.743 (n = 179, err = 34.2)
|   |   [17] latitude > 55.9396: 0.929 (n = 269, err = 17.7)

Number of inner nodes:    8
Number of terminal nodes: 9
```

```r
plot(ct3)
```




```r
mB1 <- glm(y01 ~ pconif, data = x, family = binomial)
summary(mB1)
```

```
Call:
glm(formula = y01 ~ pconif, family = binomial, data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.89412    0.09742  -9.178   <2e-16 ***
pconif       4.34123    0.33705  12.880   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1155.23  on 833  degrees of freedom
Residual deviance:  873.29  on 832  degrees of freedom
AIC: 877.29

Number of Fisher Scoring iterations: 5
```

```r
coef(mB1)
```

```
(Intercept)      pconif
```

```
 -0.8941201   4.3412307
```

```r
confint(mB1)
```

```
Waiting for profiling to be done...

                2.5 %     97.5 %
(Intercept) -1.087483 -0.7053379
pconif       3.706935  5.0307008
```

```r
anova(mB1, test = "Chi")
```

```
Analysis of Deviance Table

Model: binomial, link: logit

Response: y01

Terms added sequentially (first to last)

       Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                     833    1155.23
pconif  1   281.94       832     873.29 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
mB2 <- glm(y01 ~ pconif + I(pconif^2), data = x, family = binomial)
summary(mB2)
```

```
Call:
glm(formula = y01 ~ pconif + I(pconif^2), family = binomial,
    data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -1.2756     0.1123 -11.362   <2e-16 ***
pconif       14.6589     1.2458  11.767   <2e-16 ***
I(pconif^2) -12.4296     1.3328  -9.326   <2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1155.23  on 833  degrees of freedom
Residual deviance:  770.92  on 831  degrees of freedom
AIC: 776.92

Number of Fisher Scoring iterations: 5
```

```r
coef(mB2)
```

```
(Intercept)      pconif I(pconif^2)
  -1.275621   14.658917  -12.429593
```

```r
confint(mB2)
```

```
Waiting for profiling to be done...

                 2.5 %    97.5 %
(Intercept)  -1.500003 -1.059499
pconif       12.304248 17.199271
I(pconif^2) -15.125738 -9.886935
```

```r
anova(mB2, test = "Chi")
```

```
Analysis of Deviance Table

Model: binomial, link: logit

Response: y01

Terms added sequentially (first to last)

            Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                          833    1155.23
pconif       1   281.94       832     873.29 < 2.2e-16 ***
I(pconif^2)  1   102.37       831     770.92 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
mB3 <- glm(y01 ~ pconif + I(pconif^2) + ahm, data = x, family = binomial)
summary(mB3)
```

```
Call:
glm(formula = y01 ~ pconif + I(pconif^2) + ahm, family = binomial,
    data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  5.76730    0.76630   7.526 5.23e-14 ***
pconif       7.73123    1.29818   5.955 2.59e-09 ***
I(pconif^2) -7.34951    1.36475  -5.385 7.24e-08 ***
ahm         -0.25521    0.02961  -8.618  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1155.23  on 833  degrees of freedom
Residual deviance:  624.29  on 830  degrees of freedom
AIC: 632.29

Number of Fisher Scoring iterations: 6
```

```r
coef(mB3)
```

```
(Intercept)      pconif I(pconif^2)         ahm
  5.7673011   7.7312338  -7.3495082  -0.2552066
```

```r
confint(mB3)
```

```
Waiting for profiling to be done...

                  2.5 %     97.5 %
(Intercept)   4.3362135  7.3440792
pconif        5.2413505 10.3432783
I(pconif^2) -10.0788398 -4.7132854
ahm          -0.3167206 -0.2005264
```

```r
anova(mB3, test = "Chi")
```

```
Analysis of Deviance Table

Model: binomial, link: logit

Response: y01

Terms added sequentially (first to last)

            Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                          833    1155.23
pconif       1   281.94       832     873.29 < 2.2e-16 ***
I(pconif^2)  1   102.37       831     770.92 < 2.2e-16 ***
ahm          1   146.63       830     624.29 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
mB4 <- glm(y01 ~ pconif + I(pconif^2) + ahm + thf, data = x, family = binomial)
summary(mB4)
```

```
Call:
glm(formula = y01 ~ pconif + I(pconif^2) + ahm + thf, family = binomial,
    data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   5.9065     0.7563   7.810 5.72e-15 ***
pconif        5.7208     1.3711   4.172 3.01e-05 ***
I(pconif^2)  -5.6698     1.4108  -4.019 5.85e-05 ***
ahm          -0.2322     0.0288  -8.063 7.42e-16 ***
thf          -1.4127     0.3375  -4.186 2.84e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1155.23  on 833  degrees of freedom
Residual deviance:  606.68  on 829  degrees of freedom
AIC: 616.68
```

```
Number of Fisher Scoring iterations: 6
```

```r
coef(mB4)
```

```
(Intercept)      pconif I(pconif^2)         ahm         thf 
  5.9064674   5.7207650  -5.6697851  -0.2322369  -1.4126714 
```

```r
confint(mB4)
```

```
Waiting for profiling to be done...

                 2.5 %     97.5 %
(Intercept)  4.4966592  7.4665061
pconif       3.0762361  8.4649576
I(pconif^2) -8.4814687 -2.9346600
ahm         -0.2923873 -0.1793401
thf         -2.0787214 -0.7533104
```

```r
anova(mB4, test = "Chi")
```

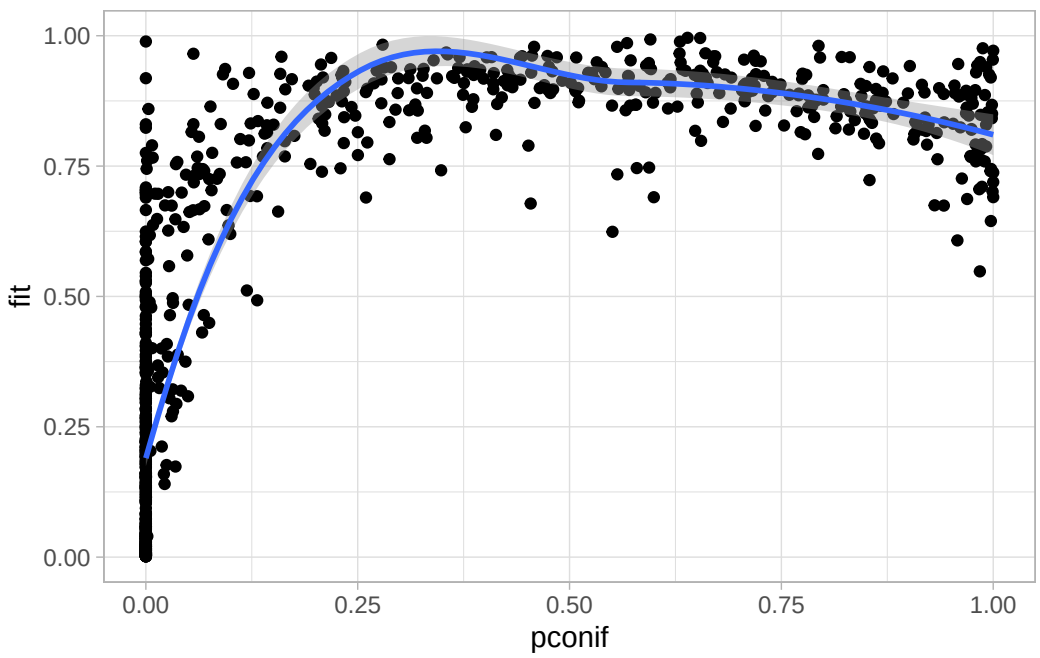
```
Analysis of Deviance Table

Model: binomial, link: logit

Response: y01

Terms added sequentially (first to last)

            Df Deviance Resid. Df Resid. Dev  Pr(>Chi)    
NULL                          833    1155.23              
pconif       1  281.944       832     873.29 < 2.2e-16 ***
I(pconif^2)  1  102.369       831     770.92 < 2.2e-16 ***
ahm          1  146.628       830     624.29 < 2.2e-16 ***
thf          1   17.609       829     606.68 2.713e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
MuMIn::model.sel(mB0, mB1, mB2, mB3, mB4)
```

```
Model selection table
     (Intrc)  pconf pconf^2     ahm    thf df   logLik   AICc  delta weight
mB4  5.90600  5.721   -5.67 -0.2322 -1.413  5 -303.340  616.8   0.00      1
mB3  5.76700  7.731   -7.35 -0.2552         4 -312.144  632.3  15.58      0
mB2 -1.27600 14.660  -12.43                 3 -385.458  776.9 160.19      0
mB1 -0.89410  4.341                         2 -436.643  877.3 260.55      0
mB0  0.06717                                1 -577.615 1157.2 540.48      0
Models ranked by AICc(x)
```

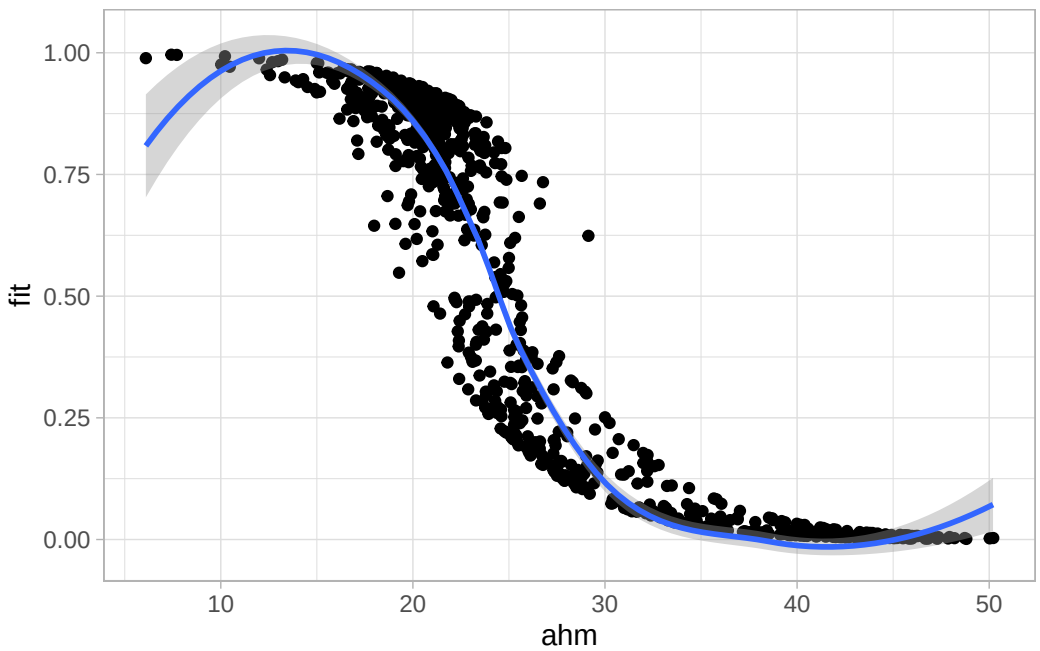
```r
# marginal effects
data.frame(x, fit = fitted(mB4)) |>
    ggplot(aes(x = pconif, y = fit)) +
    geom_jitter() +
    geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



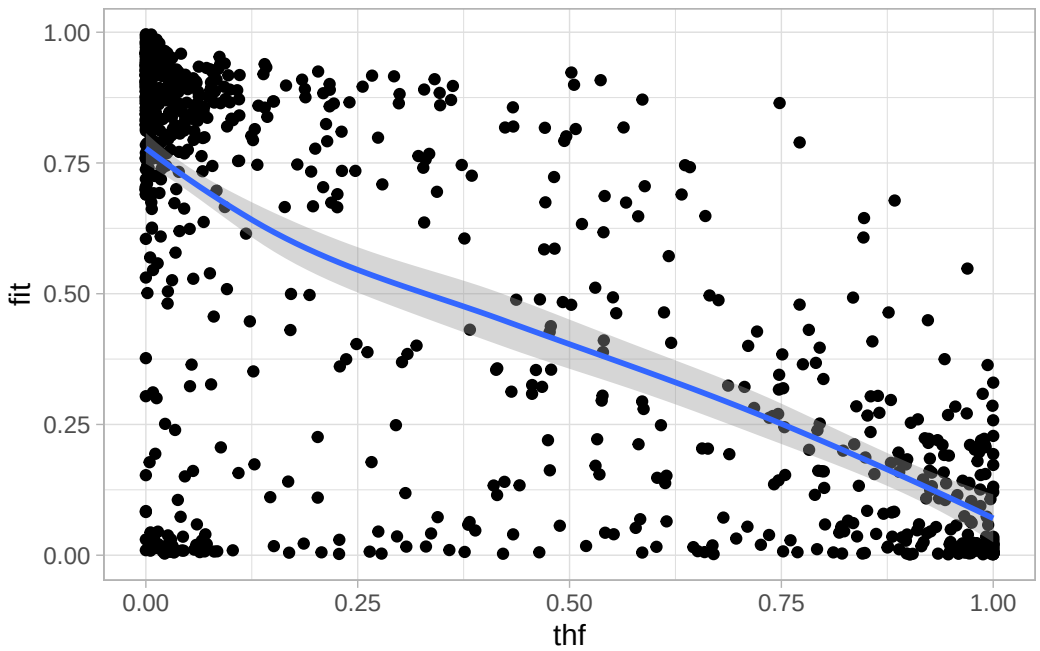
```r
data.frame(x, fit = fitted(mB4)) |>
    ggplot(aes(x = ahm, y = fit)) +
    geom_jitter() +
    geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

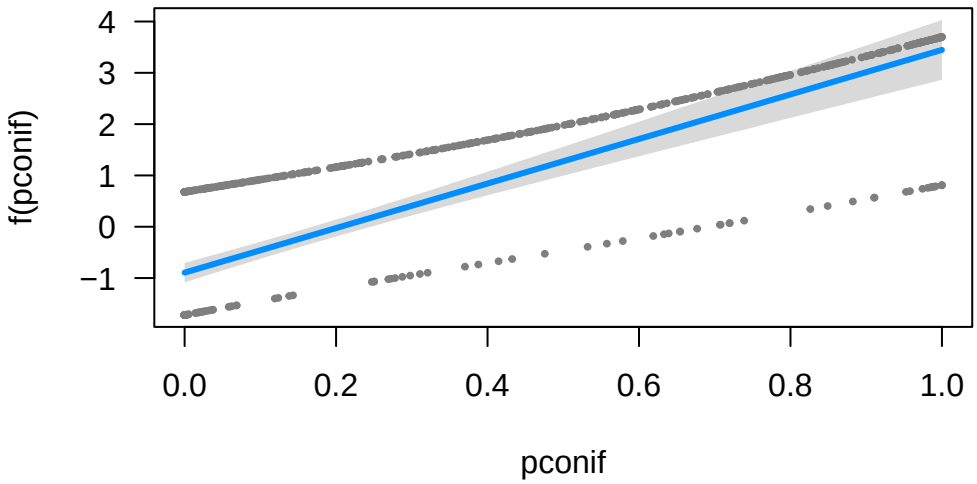



```r
data.frame(x, fit = fitted(mB4)) |>
    ggplot(aes(x = thf, y = fit)) +
    geom_jitter() +
    geom_smooth()
```

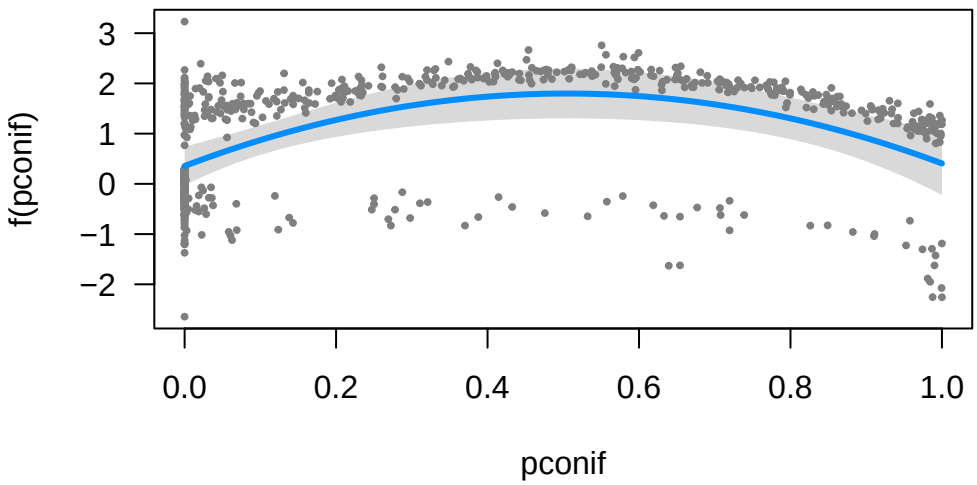
```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



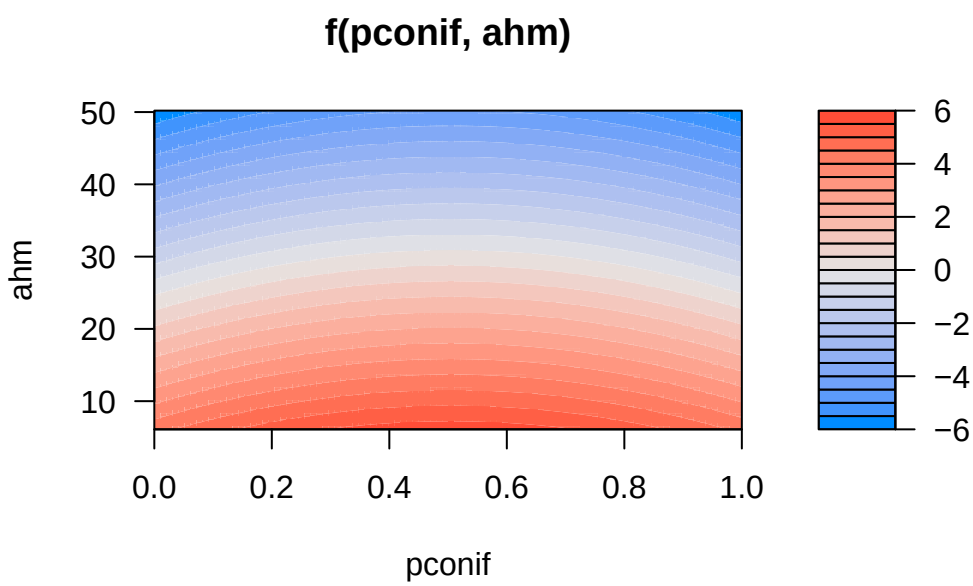

```
# partial effects
visreg::visreg(mB1, "pconif")
```




```
visreg::visreg(mB4, "pconif")
```




```
visreg::visreg2d(mB4, "pconif", "ahm")
```



Notice the following: 1. The logic behind LM and GLM is nearly identical. Our goal is to reduce the prediction uncertainty using auxiliary information. 2. The main point is that we have to use appropriate statistical model to quantify the intrinsic variability in the response variable. 3. In both LM and GLM, we model the mean as a function of the covariates. This function has to conform to the natural limits imposed by the statistical model. The mean has to be positive for positive random variables; between 0 and 1 for a binary random variable etc. 4. Concept of residuals is extremely tricky when the range of the statistical model is constrained.

Count regression models

We'll use the JOSM data set.

Assemble the JOSM data:

```
x <- detect::josm$surveys |>
    select(
        Longitude,
        Latitude,
        WindStart,
        TSSR,
        DAY,
        Open,
        Water,
        Decid,
        OpenWet,
        Conif,
        ConifWet,
        Agr,
        UrbInd,
        SoftLin,
        Roads
    )
x[is.na(x$WindStart), "WindStart"] <- c(0, 3)

x$FOR <- x$Decid + x$Conif + x$ConifWet
x$HF <- x$Agr + x$UrbInd + x$Roads + x$SoftLin
x$WET <- x$OpenWet + x$ConifWet + x$Water
cn <- c(
    "Open", "Water", "Agr", "UrbInd", "SoftLin", "Roads", "Decid",
    "OpenWet", "Conif", "ConifWet"
)
h <- mefa4::find_max(x[, cn])
x$HAB <- droplevels(h$index) # drop empty levels
x$DEC <- ifelse(x$HAB == "Decid", 1, 0)

spp <- "OVEN" # change here if you want to use another species

y <- mefa4::Xtab(~ SiteID + SpeciesID, detect::josm$counts)[, spp, drop = FALSE]
x$Count <- y[rownames(x), ]
```

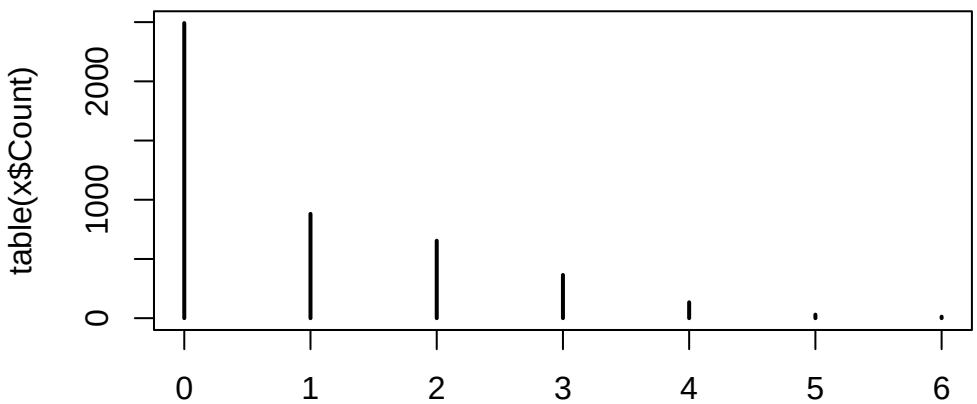
Variables are similar to the ABMI example.

Let's inspect the response variable:

```
table(x$Count)
```

```
   0    1    2    3    4    5    6 
2492  881  654  365  134   30   13 
```

```
plot(table(x$Count))
```



The distribution looks somewhat 0 inflated. What are the possible reasons for that?

- conditions lead to the absence of the species
- detected counts are lower than the actual counts

The types of models used most often to model count data and the functions used to fit them:

- Poisson: `stats::glm()`
- Negative Binomial (higher or lower variance than Poisson): `MASS::glm.nb()`
- Zero-inflated Poisson (ZIP): `pscl::zeroinfl()`
- Zero-inflated Negative Binomial (ZINB): : `pscl::zeroinfl()`

Other approaches include additive models (`mgcv::gam()`) and tree based methods (see the gbm and xgboost packages).

Let us start with the Poisson model.

Poisson null model

The null model states that the expected values of the count at all locations are identical: $E[Y_i] = \lambda$ $(i = 1, ..., n)$, where Y_i is a random variable that follows a Poisson distribution with mean λ: $(Y_i \mid \lambda) \sim Poisson(\lambda)$. The observation (y_i) is a realization of the random variables Y at site i, these observations are independent and identically distributed (i.i.d.), and we have n observations in total.

Saying the the distribution is Poisson is an assumption in itself. For example we assume that the variance equals the mean $(V(\mu) = \mu)$.

```
mP0 <- glm(Count ~ 1, data = x, family = poisson)
```

The `family=poisson` specification implicitly assumes that we use a logarithmic link functions, that is to say that $log(\lambda) = \beta_0$, or equivalently: $\lambda = e^{\beta_0}$. The mean of the observations equal the mean of the fitted values, as expected:

```
mean(mP0$y)
```

```
[1] 0.8859707
```

```
mean(fitted(mP0))
```

```
[1] 0.8859707
```

```
exp(coef(mP0))
```

```
(Intercept)
  0.8859707
```

The logarithmic function is called the link function, its inverse, the exponential function is called the inverse link function. The model family has these conveniently stored for us:

```
mP0$family
```

```
Family: poisson
Link function: log
```

```r
mP0$family$linkfun
```

```
function (mu)
log(mu)
<environment: namespace:stats>
```

```r
mP0$family$linkinv
```

```
function (eta)
pmax(exp(eta), .Machine$double.eps)
<environment: namespace:stats>
```

Inspect the summary

```r
summary(mP0)
```

```
Call:
glm(formula = Count ~ 1, family = poisson, data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.12107    0.01572  -7.703 1.33e-14 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 7447.9  on 4568  degrees of freedom
Residual deviance: 7447.9  on 4568  degrees of freedom
AIC: 12603

Number of Fisher Scoring iterations: 6
```

Notice that the residual deviance much higher than residual degrees of freedom. This indicates that our parametric model (Poisson error distribution, constant expected value) is not quite right. See if we can improve that somehow and explain more of the variation.

We can pick an error distribution that would fit the residuals around the constant expected value better (i.e. using random effects). But this way we would not learn about what is driving

the variation in the counts. We would also have a really hard time predicting abundance of the species for unsurveyed locations. We would be right on average, but we wouldn't be able to tell how abundance varies along e.g. a disturbance gradient or with tree cover.

An alternative approach would be to find predictors that could explain the variation.

Main effects

We fit a parametric (Poisson) linear model using `Decid` as a predictor:

```
mP1 <- glm(Count ~ Decid, data = x, family = poisson)
mean(mP1$y)
```

```
[1] 0.8859707
```

```
mean(fitted(mP1))
```

```
[1] 0.8859707
```

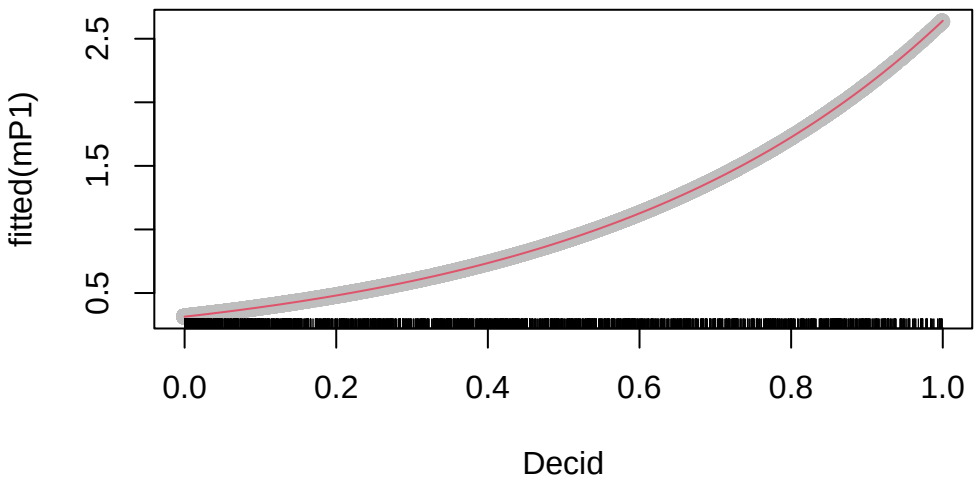
```
coef(mP1)
```

```
(Intercept)       Decid
  -1.158833    2.130040
```

Same as before, the mean of the observations equal the mean of the fitted values. But instead of only the intercept, now we have 2 coefficients estimated. Our linear predictor thus looks like: $log(\lambda_i) = \beta_0 + \beta_1 x_{1i}$. This means that expected abundance is e^{β_0} where `Decid`=0, $e^{\beta_0} e^{\beta_1}$ where `Decid`=1, and $e^{\beta_0 + \beta_1 x_1}$ in between.

The relationship can be visualized by plotting the fitted values against the predictor, or using the coefficients to make predictions using our formula:

```
## make a sequence between 0 and 1
dec <- seq(from = 0, to = 1, by = 0.01)
## predict lambda
lam <- exp(coef(mP1)[1] + coef(mP1)[2] * dec)

plot(fitted(mP1) ~ Decid, x, pch = 19, col = "grey") # fitted
lines(lam ~ dec, col = 2) # our predicted
rug(x$Decid) # observed x values
```



The model summary tells us that residuals are not quite right The residual deviance is still higher than residual degrees of freedom (these should be close if the Poisson assumption holds, but it is much better than what we saw for the null model).

We also learned that the `Decid` effect is significant (meaning that the effect size is large compared to the standard error):

```
summary(mP1)
```

```
Call:
glm(formula = Count ~ Decid, family = poisson, data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.15883    0.03507  -33.04   <2e-16 ***
Decid        2.13004    0.05359   39.75   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 7447.9  on 4568  degrees of freedom
Residual deviance: 5760.2  on 4567  degrees of freedom
AIC: 10917

Number of Fisher Scoring iterations: 6
```

Note: we see a significant (<0.05) P-value for the intercept as well. It is totally meaningless. That P-value relates to the hull hypothesis of the intercept (β_0) being 0. There is nothing special about that, it is like saying the average abundance is different from 1.

But when β_1 is significantly different from 0, it means that the main effect has non-negligible effect on the mean abundance.

We can compare this model to the null (constant, intercept-only) model:

```
AIC(mP0, mP1)
```

```
    df      AIC
mP0  1 12602.84
mP1  2 10917.16
```

```
BIC(mP0, mP1)
```

```
    df      BIC
mP0  1 12609.27
mP1  2 10930.02
```

```
MuMIn::model.sel(mP0, mP1)
```

```
Model selection table
    (Intrc) Decid df    logLik    AICc   delta weight
mP1 -1.1590  2.13  2 -5456.581 10917.2    0.00      1
mP0 -0.1211        1 -6300.422 12602.8 1685.68      0
Models ranked by AICc(x)
```

AIC uses the negative log likelihood and the number of parameters as penalty. Smaller value indicate a model that is closer to the (unknowable) true model (caveat: this statement is true only asymptotically, i.e. it holds for large sample sizes). For small samples, we often use BIC (more penalty for complex models when sample size is small), or AICc (as in `MuMIn::model.sel()`).

Non-linear effects

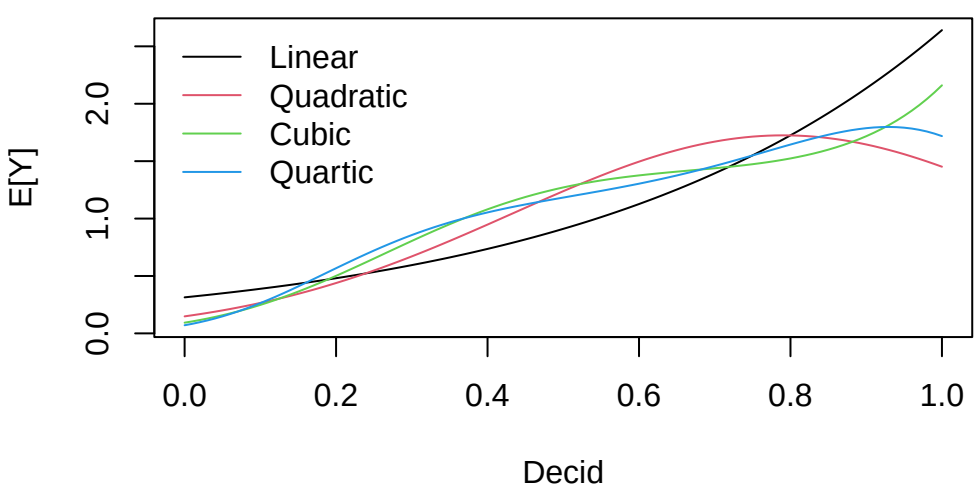
We can use polynomial terms to capture non (log) linear effects:

```
mP12 <- glm(Count ~ Decid + I(Decid^2), data = x, family = poisson)
mP13 <- glm(Count ~ Decid + I(Decid^2) + I(Decid^3), data = x, family = poisson)
mP14 <- glm(Count ~ Decid + I(Decid^2) + I(Decid^3) + I(Decid^4), data = x, family = poisson
MuMIn::model.sel(mP0, mP1, mP12, mP13, mP14)
```

```
Model selection table
     (Intrc)  Decid Decid^2 Decid^3 Decid^4 df    logLik    AICc   delta weight
mP14 -2.6470 16.610 -38.740   41.78  -16.47  5 -5232.629 10475.3    0.00      1
mP13 -2.3730 11.340 -16.250    8.06          4 -5243.979 10496.0   20.70      0
mP12 -1.9090  6.209  -3.927                  3 -5286.517 10579.0  103.77      0
mP1  -1.1590  2.130                          2 -5456.581 10917.2  441.89      0
mP0  -0.1211                                 1 -6300.422 12602.8 2127.57      0
Models ranked by AICc(x)
```

Not a surprise that the most complex model won, we had enough degrees of freedoms to spare.

```r
xnew <- data.frame(Decid = seq(0, 1, 0.01))

pr <- cbind(
    predict(mP1, xnew, type = "response"),
    predict(mP12, xnew, type = "response"),
    predict(mP13, xnew, type = "response"),
    predict(mP14, xnew, type = "response")
)
matplot(xnew$Decid, pr,
    lty = 1, type = "l",
    xlab = "Decid", ylab = "E[Y]"
)
legend("topleft",
    lty = 1, col = 1:4, bty = "n",
    legend = c("Linear", "Quadratic", "Cubic", "Quartic")
)
```



Categorical variables

Categorical variables are expanded into a *model matrix* before parameter estimation. The model matrix usually contains indicator variables for each level (value 1 when factor value equals a particular label, 0 otherwise) except for the *reference category* (check `relevel` if you want to change the reference category).

The estimate for the reference category comes from the intercept, the rest of the estimates are relative to the reference category. In the log-linear model example this means a ratio.

```
head(model.matrix(~DEC, x))
```

```
        (Intercept) DEC
CL10102           1   1
CL10106           1   0
CL10108           1   0
CL10109           1   1
CL10111           1   1
CL10112           1   1
```

```
mP2 <- glm(Count ~ DEC, data = x, family = poisson)
summary(mP2)
```

```
Call:
glm(formula = Count ~ DEC, family = poisson, data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.85106    0.03070  -27.72   <2e-16 ***
DEC          1.21104    0.03574   33.89   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 7447.9  on 4568  degrees of freedom
Residual deviance: 6123.1  on 4567  degrees of freedom
AIC: 11280

Number of Fisher Scoring iterations: 6
```

```r
coef(mP2)
```

```
(Intercept)         DEC
 -0.8510608   1.2110412
```

The estimate for a non-deciduous landscape is e^{β_0}, and it is $e^{\beta_0}e^{\beta_1}$ for deciduous landscapes. (Of course such binary classification at the landscape (1 km^2) level doesn't really makes sense.)

```r
MuMIn::model.sel(mP1, mP2)
```

```
Model selection table
    (Intrc) Decid   DEC df    logLik    AICc  delta weight
mP1 -1.1590  2.13        2 -5456.581 10917.2   0.00      1
mP2 -0.8511       1.211  2 -5638.039 11280.1 362.92      0
Models ranked by AICc(x)
```

Having estimates for each land cover type improves the model, but the model using continuous variable is still better

```r
mP3 <- glm(Count ~ HAB, data = x, family = poisson)
summary(mP3)
```

```
Call:
glm(formula = Count ~ HAB, family = poisson, data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -1.3863     0.5774  -2.401   0.0163 *
HABWater      1.0296     0.6901   1.492   0.1357
HABAgr        0.6931     0.9129   0.759   0.4477
HABUrbInd     0.1335     0.7638   0.175   0.8612
HABRoads    -10.9163   201.2853  -0.054   0.9567
HABDecid      1.7463     0.5776   3.023   0.0025 **
HABOpenWet    0.4220     0.5914   0.714   0.4755
HABConif      0.9214     0.5792   1.591   0.1117
HABConifWet   0.2942     0.5790   0.508   0.6114
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 7447.9  on 4568  degrees of freedom
Residual deviance: 6023.2  on 4560  degrees of freedom
AIC: 11194

Number of Fisher Scoring iterations: 10
```

```r
MuMIn::model.sel(mP1, mP2, mP3)
```

```
Model selection table
    (Intrc) Decid   DEC HAB df    logLik    AICc  delta weight
mP1 -1.1590  2.13            2 -5456.581 10917.2   0.00      1
mP3 -1.3860               +  9 -5588.059 11194.2 276.99      0
mP2 -0.8511       1.211      2 -5638.039 11280.1 362.92      0
Models ranked by AICc(x)
```

The prediction in this case would look like: $log(\lambda_i) = \beta_0 + \sum_{j=1}^{k-1} \beta_j x_{ji}$, where we have k factor levels (and $k-1$ indicator variables besides the intercept).

Here is a general way of calculating fitted values or making predictions based on the design matrix (`X`) and the coefficients (`b`) (column ordering in `X` must match the elements in `b`) given a parametric log-linear model `object` and data frame `df` (the code won't run as is, `object` is just a placeholder for your GLM model object):

```r
b <- coef(object)
X <- model.matrix(formula(object), df)
exp(X %*% b)
```

Multiple main effects

We can add main effects by providing a new formula to a new model. Or we can use the `update()` function. The `. ~ .` means to keep both sides of the formula but add (`+`) or remove (`-`) terms from the original scope. Note: the `update` function can be used to update any function call, not just GLM models.

```r
# mP4 <- glm(Count ~ Decid + ConifWet, data=x, family=poisson)
mP4 <- update(mP1, . ~ . + ConifWet)
summary(mP4)
```

```
Call:
glm(formula = Count ~ Decid + ConifWet, family = poisson, data = x)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.69255    0.05547 -12.485   <2e-16 ***
Decid        1.61480    0.07169  22.525   <2e-16 ***
ConifWet    -0.98628    0.09906  -9.957   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 7447.9  on 4568  degrees of freedom
Residual deviance: 5659.7  on 4566  degrees of freedom
AIC: 10819

Number of Fisher Scoring iterations: 6
```

```r
MuMIn::model.sel(mP0, mP1, mP4)
```

```
Model selection table
    (Intrc) Decid   CnfWt df    logLik    AICc   delta weight
mP4 -0.6925 1.615 -0.9863  3 -5406.310 10818.6    0.00      1
mP1 -1.1590 2.130          2 -5456.581 10917.2   98.54      0
mP0 -0.1211                1 -6300.422 12602.8 1784.22      0
Models ranked by AICc(x)
```

Here are some functions that can automate model selection:

- `drop1`: evaluate which variable to drop to lower AIC the most
- `add1`: evaluate which variable to add to lower AIC the most
- `step`: perform forward/backward model selection using `add1`/`drop1`

Interactions

When we consider interactions between two variables (say x_1 and x_2), we refer to adding another variable to the model matrix that is a product of the two variables ($x_{12} = x_1 x_2$):

```r
head(model.matrix(~ x1 * x2, data.frame(x1 = 1:4, x2 = 10:7)))
```

```
  (Intercept) x1 x2 x1:x2
1           1  1 10    10
2           1  2  9    18
3           1  3  8    24
4           1  4  7    28
```

Let's consider interaction between our two predictors from before:

```r
mP5 <- glm(Count ~ Decid * ConifWet, data = x, family = poisson)
summary(mP5)
```

```
Call:
glm(formula = Count ~ Decid * ConifWet, family = poisson, data = x)

Coefficients:
               Estimate Std. Error z value Pr(>|z|)
(Intercept)    -0.55157    0.05643  -9.775   <2e-16 ***
Decid           1.20486    0.07799  15.448   <2e-16 ***
ConifWet       -2.32015    0.14875 -15.598   <2e-16 ***
Decid:ConifWet  5.34691    0.35615  15.013   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 7447.9  on 4568  degrees of freedom
Residual deviance: 5415.8  on 4565  degrees of freedom
AIC: 10577

Number of Fisher Scoring iterations: 6
```

```r
MuMIn::model.sel(mP0, mP1, mP4, mP5)
```

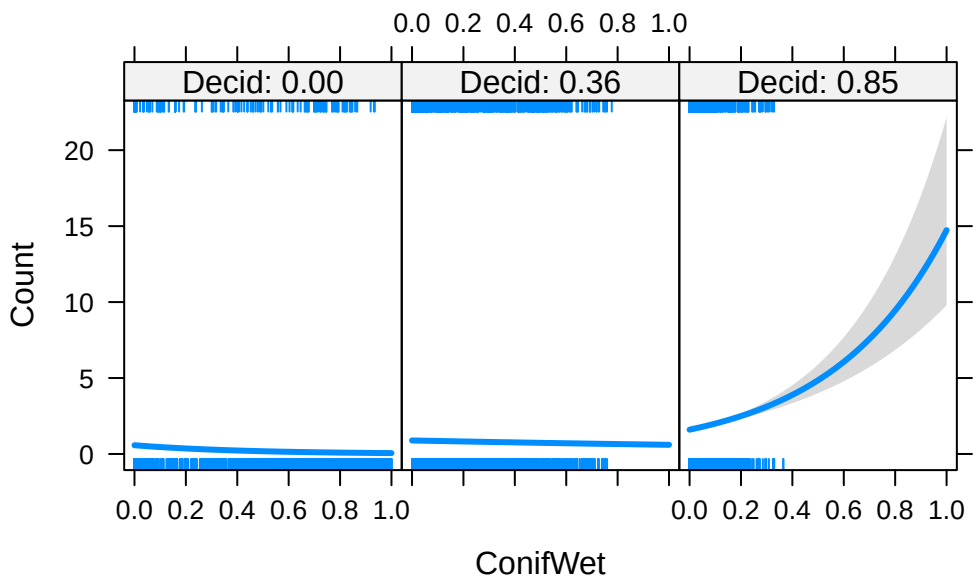
```
Model selection table
      (Int)   Dcd     CnW CnW:Dcd df    logLik    AICc   delta weight
mP5 -0.5516 1.205 -2.3200   5.347  4 -5284.389 10576.8    0.00      1
mP4 -0.6925 1.615 -0.9863          3 -5406.310 10818.6  241.84      0
```

```
mP1 -1.1590 2.130                    2 -5456.581 10917.2  340.38      0
mP0 -0.1211                          1 -6300.422 12602.8 2026.06      0
Models ranked by AICc(x)
```

The model with the interaction is best supported, but how do we make sense of this relationship? We can't easily visualize it in a single plot. We can either

1. fix all variables (at their mean/meadian) and see how the response is changing along a single variable: this is called a *conditional* effect (conditional on fixing other variables), this is what `visreg::visreg()` does
2. or plot the fitted values against the predictor variable (one at a time), this is called a *marginal* effect, and this is what `ResourceSelection::mep()` does

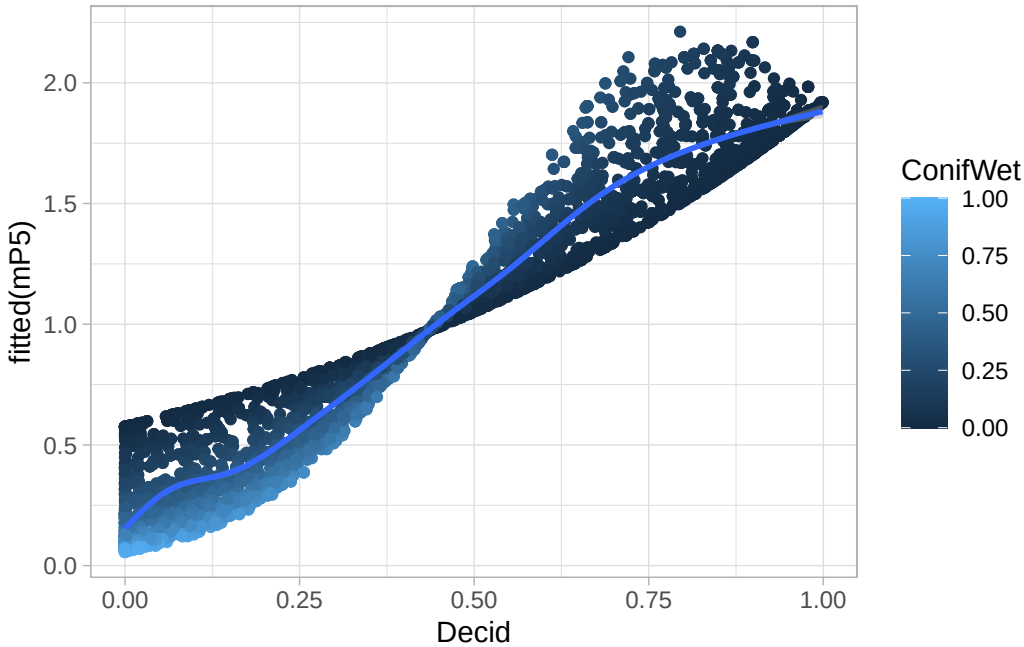
```r
visreg::visreg(mP5, scale = "response", xvar = "ConifWet", by = "Decid")
```




```r
ggplot(x, aes(x = Decid, y = fitted(mP5), col = ConifWet)) +
    geom_point() +
    geom_smooth() +
    theme_light()
```

```
`geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'

Warning: The following aesthetics were dropped during statistical transformation:
colour.
```

```
i This can happen when ggplot fails to infer the correct grouping structure in
  the data.
i Did you forget to specify a `group` aesthetic or to convert a numerical
  variable into a factor?
```



Final battle of Poisson models:

```
MuMIn::model.sel(mP0, mP1, mP12, mP13, mP14, mP2, mP3, mP4, mP5)
```

```
Model selection table
       (Int)    Dcd   Dcd^2 Dcd^3  Dcd^4   DEC HAB     CnW CnW:Dcd df    logLik
mP14 -2.6470 16.610 -38.740 41.78 -16.47                             5 -
5232.629
mP13 -2.3730 11.340 -16.250  8.06                                    4 -
5243.979
mP5  -0.5516  1.205                                -2.3200   5.347  4 -
5284.389
mP12 -1.9090  6.209  -3.927                                          3 -
5286.517
mP4  -0.6925  1.615                                -0.9863          3 -
5406.310
mP1  -1.1590  2.130                                                  2 -
5456.581
```

```
mP3  -1.3860                                        +                  9 -
5588.059
mP2  -0.8511                              1.211                        2 -
5638.039
mP0  -0.1211                                                           1 -
6300.422
        AICc   delta weight
mP14 10475.3    0.00      1
mP13 10496.0   20.70      0
mP5  10576.8  101.52      0
mP12 10579.0  103.77      0
mP4  10818.6  343.35      0
mP1  10917.2  441.89      0
mP3  11194.2  718.89      0
mP2  11280.1  804.81      0
mP0  12602.8 2127.57      0
Models ranked by AICc(x)
```

Of course, the most complex model wins but the Chi-square test is still significant (indicating lack of fit). Let's try different error distribution.

Different error distributions

We will use the 2-variable model with interaction:

```r
mP <- glm(Count ~ Decid * ConifWet, data = x, family = poisson)
summary(mP)
```

```
Call:
glm(formula = Count ~ Decid * ConifWet, family = poisson, data = x)

Coefficients:
               Estimate Std. Error z value Pr(>|z|)
(Intercept)    -0.55157    0.05643  -9.775   <2e-16 ***
Decid           1.20486    0.07799  15.448   <2e-16 ***
ConifWet       -2.32015    0.14875 -15.598   <2e-16 ***
Decid:ConifWet  5.34691    0.35615  15.013   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 7447.9  on 4568  degrees of freedom
Residual deviance: 5415.8  on 4565  degrees of freedom
AIC: 10577

Number of Fisher Scoring iterations: 6
```

Let us try the Negative Binomial distribution first. This distribution is related to Binomial experiments (number of trials required to get a fixed number of successes given a binomial probability). It can also be derived as a mixture of Poisson and Gamma distributions (see Wikipedia), which is a kind of hierarchical model. In this case, the Gamma distribution acts as an i.i.d. random effect for the intercept: $Y_i \sim Poisson(\lambda_i)$, $\lambda_i \sim Gamma(e^{\beta_0+\beta_1 x_{1i}}, \gamma)$, where γ is the Gamma variance.

The Negative Binomial variance (using the parametrization common in R functions) is a function of the mean and the scale: $V(\mu) = \mu + \mu^2/\theta$.

```r
mNB <- MASS::glm.nb(Count ~ Decid * ConifWet, data = x)
summary(mNB)
```

```
Call:
MASS::glm.nb(formula = Count ~ Decid * ConifWet, data = x, init.theta = 3.524756181,
    link = log)

Coefficients:
               Estimate Std. Error z value Pr(>|z|)
(Intercept)    -0.58101    0.06291  -9.236   <2e-16 ***
Decid           1.23695    0.08919  13.868   <2e-16 ***
ConifWet       -2.36485    0.16040 -14.743   <2e-16 ***
Decid:ConifWet  5.70380    0.40132  14.212   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(3.5248) family taken to be 1)

    Null deviance: 6086.2  on 4568  degrees of freedom
Residual deviance: 4388.4  on 4565  degrees of freedom
AIC: 10464

Number of Fisher Scoring iterations: 1
```

```
              Theta:  3.525
          Std. Err.:  0.411

 2 x log-likelihood:  -10453.887
```

Next, we look at zero-inflated models. In this case, the mixture distribution is a Bernoulli distribution and a count distribution (Poisson or Negative Binomial, for example). The 0's can come from both the zero and the count distributions, whereas the >0 values can only come from the count distribution: $A_i \sim Bernoulli(\varphi)$, $Y_i \sim Poisson(A_i \lambda_i)$.

The zero part of the zero-inflated models are often parametrized as probability of zero $(1 - \varphi)$, as in the `pscl::zeroinfl` function:

```r
## Zero-inflated Poisson
mZIP <- pscl::zeroinfl(Count ~ Decid * ConifWet | 1, x, dist = "poisson")
summary(mZIP)
```

```
Call:
pscl::zeroinfl(formula = Count ~ Decid * ConifWet | 1, data = x, dist = "poisson")

Pearson residuals:
    Min      1Q  Median      3Q     Max 
-1.2160 -0.7011 -0.3391  0.3766  8.9394 

Count model coefficients (poisson with log link):
               Estimate Std. Error z value Pr(>|z|)    
(Intercept)    -0.31127    0.06503  -4.787 1.69e-06 ***
Decid           1.05891    0.08588  12.330  < 2e-16 ***
ConifWet       -2.45044    0.15622 -15.686  < 2e-16 ***
Decid:ConifWet  5.94422    0.38368  15.493  < 2e-16 ***

Zero-inflation model coefficients (binomial with logit link):
            Estimate Std. Error z value Pr(>|z|)    
(Intercept)  -1.5989     0.1021  -15.65   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Number of iterations in BFGS optimization: 12 
Log-likelihood: -5218 on 5 Df
```

```r
## Zero-inflated Negative Binomial
mZINB <- pscl::zeroinfl(Count ~ Decid * ConifWet | 1, x, dist = "negbin")
summary(mZINB)
```

```
Call:
pscl::zeroinfl(formula = Count ~ Decid * ConifWet | 1, data = x, dist = "negbin")

Pearson residuals:
    Min      1Q  Median      3Q     Max 
-1.1874 -0.6902 -0.3375  0.3616  8.9455 

Count model coefficients (negbin with log link):
               Estimate Std. Error z value Pr(>|z|)    
(Intercept)    -0.37625    0.07312  -5.146 2.66e-07 ***
Decid           1.10981    0.09105  12.189  < 2e-16 ***
ConifWet       -2.43272    0.15872 -15.327  < 2e-16 ***
Decid:ConifWet  5.93085    0.39641  14.962  < 2e-16 ***
Log(theta)      2.61874    0.51590   5.076 3.85e-07 ***

Zero-inflation model coefficients (binomial with logit link):
            Estimate Std. Error z value Pr(>|z|)    
(Intercept)  -1.8513     0.1917  -9.655   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Theta = 13.7184 
Number of iterations in BFGS optimization: 22 
Log-likelihood: -5215 on 6 Df
```

```r
## Hurdle Poisson
mHP <- pscl::hurdle(Count ~ Decid * ConifWet | 1, x, dist = "poisson")
summary(mHP)
```

```
Call:
pscl::hurdle(formula = Count ~ Decid * ConifWet | 1, data = x, dist = "poisson")

Pearson residuals:
    Min      1Q  Median      3Q     Max 
-0.7984 -0.7485 -0.7342  0.7585  5.1575 
```

```
Count model coefficients (truncated poisson with log link):
               Estimate Std. Error z value Pr(>|z|)
(Intercept)     0.19488    0.08134   2.396 0.016577 *
Decid           0.39188    0.11078   3.538 0.000404 ***
ConifWet       -0.99332    0.25202  -3.941 8.10e-05 ***
Decid:ConifWet  2.34801    0.57730   4.067 4.76e-05 ***
Zero hurdle model coefficients (binomial with logit link):
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.18216    0.02971  -6.131 8.73e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Number of iterations in BFGS optimization: 10
Log-likelihood: -5819 on 5 Df
```

```r
## Hurdle Negative Binomial
mHNB <- pscl::hurdle(Count ~ Decid * ConifWet | 1, x, dist = "negbin")
```

```
Warning in sqrt(diag(vc_count)[kx + 1]): NaNs produced
```

```r
summary(mHNB)
```

```
Call:
pscl::hurdle(formula = Count ~ Decid * ConifWet | 1, data = x, dist = "negbin")

Pearson residuals:
    Min      1Q  Median      3Q     Max
-0.7984 -0.7485 -0.7342  0.7587  5.1575

Count model coefficients (truncated negbin with log link):
               Estimate Std. Error z value Pr(>|z|)
(Intercept)     0.19499    0.08133   2.397 0.016512 *
Decid           0.39184    0.11077   3.537 0.000404 ***
ConifWet       -0.99290    0.25200  -3.940 8.14e-05 ***
Decid:ConifWet  2.34571    0.57729   4.063 4.84e-05 ***
Log(theta)     12.79005        NaN     NaN      NaN
Zero hurdle model coefficients (binomial with logit link):
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.18216    0.02971  -6.131 8.73e-10 ***
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Theta: count = 358631.9569
Number of iterations in BFGS optimization: 35
Log-likelihood: -5819 on 6 Df
```

Exercise: compare ZI and Hurdle coefficients. Anything surprising? What do the coefficients for the zero part of the model mean?

Now we compare the four different parametric models:

```
MuMIn::model.sel(mP, mNB, mZIP, mZINB, mHP, mHNB)
```

```
Model selection table
        (Int)    CnW   Dcd CnW:Dcd cnt_(Int) cnt_CnW cnt_Dcd cnt_CnW:Dcd
mZINB                                -0.3762 -2.4330  1.1100           +
mZIP                                 -0.3113 -2.4500  1.0590           +
mNB   -0.5810 -2.365 1.237   5.704
mP    -0.5516 -2.320 1.205   5.347
mHP                                   0.1949 -0.9933  0.3919           +
mHNB                                  0.1950 -0.9929  0.3918           +
      zer_(Int)         family    class init.theta link dist df    logLik
mZINB   -1.8510         b(lgt) zeroinfl                 ngbn  6 -5215.281
mZIP    -1.5990         b(lgt) zeroinfl                 pssn  5 -5217.698
mNB             NB(3.5248,log)   negbin       3.52  log       5 -5226.943
mP                      p(log)      glm                       4 -5284.389
mHP     -0.1822             ()   hurdle                 pssn  5 -5818.902
mHNB    -0.1822             ()   hurdle                 ngbn  6 -5818.902
         AICc   delta weight
mZINB 10442.6    0.00  0.804
mZIP  10445.4    2.83  0.196
mNB   10463.9   21.32  0.000
mP    10576.8  134.21  0.000
mHP   11647.8 1205.24  0.000
mHNB  11649.8 1207.24  0.000
Abbreviations:
 family: b(lgt) = 'binomial(logit)',
         NB(3.5248,log) = 'Negative Binomial(3.5248,log)',
         p(log) = 'poisson(log)'
 dist:   ngbn = 'negbin', pssn = 'poisson'
Models ranked by AICc(x)
```

Our best model is the ZINB. The probability of observing a zero as part of the zero distribution is back transformed from the zero coefficient using the inverse logit function:

```
unname(plogis(coef(mZIP, "zero"))) # P of 0 (not 1!)
```

```
[1] 0.1681348
```

Mixed models

It is also common practice to consider generalized linear mixed models (GLMMs) for count data. These mixed models are usually considered as Poisson-Lognormal mixtures. The simplest, so called i.i.d., case is similar to the Negative Binomial, but instead of Gamma, we have Lognormal distribution: $Y_i \sim Poisson(\lambda_i)$, $log(\lambda_i) = \beta_0 + \beta_1 x_{1i} + \epsilon_i$, $\epsilon_i \sim Normal(0, \sigma^2)$, where σ^2 is the Lognormal variance on the log scale.

We can use the `lme4::glmer` function: use `SiteID` as random effect (we have exactly n random effects).

```
mPLN1 <- lme4::glmer(Count ~ Decid * ConifWet + (1 | SiteID),
    data = data.frame(SiteID = rownames(x), x), family = poisson
)
summary(mPLN1)
```

```
Generalized linear mixed model fit by maximum likelihood (Laplace
  Approximation) [glmerMod]
 Family: poisson  ( log )
Formula: Count ~ Decid * ConifWet + (1 | SiteID)
   Data: data.frame(SiteID = rownames(x), x)

      AIC       BIC    logLik -2*log(L)  df.resid
  10445.9   10478.0   -5218.0   10435.9      4564

Scaled residuals:
    Min      1Q  Median      3Q     Max
-1.1475 -0.6291 -0.2855  0.4156  5.4234

Random effects:
 Groups Name        Variance Std.Dev.
 SiteID (Intercept) 0.2988   0.5466
Number of obs: 4569, groups:  SiteID, 4569
```

```
Fixed effects:
               Estimate Std. Error z value Pr(>|z|)
(Intercept)    -0.74536    0.06743  -11.05   <2e-16 ***
Decid           1.27742    0.09201   13.88   <2e-16 ***
ConifWet       -2.34745    0.16248  -14.45   <2e-16 ***
Decid:ConifWet  5.63872    0.41229   13.68   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:
            (Intr) Decid  ConfWt
Decid       -0.895
ConifWet    -0.621  0.644
Decid:CnfWt  0.176 -0.379 -0.700
```

Note: the number of unknowns we have to somehow estimate is now more than the number of observations we have. How is that possible?

Alternatively, we can use `SurveyArea` as a grouping variable. We have now $m < n$ random effects, and survey areas can be seen as larger landscapes within which the sites are clustered: $Y_i j \sim Poisson(\lambda_i j)$, $log(\lambda_i j) = \beta_0 + \beta_1 x_{1ij} + \epsilon_i$, $\epsilon_i \sim Normal(0, \sigma^2)$. The index i ($i = 1, ..., m$) defines the cluster (survey area), the j ($j = 1, ..., n_i$) defines the sites within survey area i ($n = \sum_{i=1}^{m} n_i$).

```r
mPLN2 <- lme4::glmer(Count ~ Decid * ConifWet + (1 | SurveyArea),
    data = data.frame(SurveyArea = detect::josm$surveys$SurveyArea, x),
    family = poisson
)
summary(mPLN2)
```

```
Generalized linear mixed model fit by maximum likelihood (Laplace
  Approximation) [glmerMod]
 Family: poisson  ( log )
Formula: Count ~ Decid * ConifWet + (1 | SurveyArea)
   Data: data.frame(SurveyArea = detect::josm$surveys$SurveyArea, x)

      AIC       BIC    logLik -2*log(L)  df.resid
  10047.2   10079.3   -5018.6   10037.2      4564

Scaled residuals:
    Min      1Q  Median      3Q     Max
-1.7379 -0.6434 -0.3200  0.3580  6.4556
```

```
Random effects:
 Groups     Name        Variance Std.Dev.
 SurveyArea (Intercept) 0.2934   0.5417
Number of obs: 4569, groups:  SurveyArea, 271

Fixed effects:
               Estimate Std. Error z value Pr(>|z|)
(Intercept)    -0.73773    0.07807  -9.449   <2e-16 ***
Decid           1.19195    0.09822  12.136   <2e-16 ***
ConifWet       -2.32770    0.16830 -13.830   <2e-16 ***
Decid:ConifWet  5.53058    0.39726  13.922   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:
            (Intr) Decid  ConfWt
Decid       -0.808
ConifWet    -0.609  0.628
Decid:CnfWt  0.163 -0.325 -0.670
```

In the battle of distributions (keeping the linear predictor part the same) the clustered GLMM was best supported:

```r
tmp <- AIC(mP, mNB, mZIP, mZINB, mHP, mHNB, mPLN1, mPLN2)
tmp$delta_AIC <- tmp$AIC - min(tmp$AIC)
tmp[order(tmp$AIC), ]
```

```
      df      AIC delta_AIC
mPLN2  5 10047.19    0.0000
mZINB  6 10442.56  395.3703
mZIP   5 10445.40  398.2047
mPLN1  5 10445.91  398.7171
mNB    5 10463.89  416.6952
mP     4 10576.78  529.5872
mHP    5 11647.80 1600.6119
mHNB   6 11649.80 1602.6123
```

Summary

- Quantification of uncertainty is the hallmark of statistical inference.

- Prediction uncertainty has two major components: Estimation uncertainty and Intrinsic variability.
- Estimation uncertainty can be reduced by increasing the sample size and good design.
- Intrinsic variability is usually the substantial part of the prediction uncertainty.
- Intrinsic variability can be reduced by using auxiliary information through regression modelling.
- One should use appropriate statistical model to model intrinsic variability.